

Analisi statica e verifica del software

Simone Colli, Manuel Di Agostino

`simone.colli@studenti.unipr.it`, `manuel.diagostino@studenti.unipr.it`

Appunti del corso tenuto dal **Prof. Vincenzo Arceri**

Università degli Studi di Parma
Anno Accademico 2025/2026

Indice

1	Introduzione	6
1.1	Cos'è l'affidabilità del software?	6
1.2	Perché l'affidabilità del software è importante?	6
1.3	Validazione e Verifica	7
1.4	Verifica del Software	7
2	Background matematico	10
2.1	Set notation	10
2.2	Partial order	10
2.3	Powerset	11
2.4	Poset	12
2.5	Diagramma di Hasse	12
2.6	Upper e lower bounds	13
2.7	Supremum e Infimum	14
2.8	Proprietà di lub e glb	14
2.9	Reticolo	15
2.10	Set lattice	15
2.11	Reticolo completo	16
2.12	Relations	18
2.13	Functions (or maps)	19
2.13.1	Monotonia, Embedding e Isomorfismo	19
2.13.2	Funzioni che preservano join e meet	21
2.14	Chains	21
2.14.1	Ascending Chain Condition (ACC)	22
2.15	Mappe continue	23
2.16	Fixpoint	23
2.16.1	Iterazioni di una funzione	23
2.17	Teoremi di Punto Fisso	24
3	Modellazione dei programmi	26
3.1	Poset nell'analisi statica	26
3.2	Punti fissi	27
3.3	IMP	28
3.3.1	Sintassi	28
3.3.2	Semantica concreta	28
3.4	Semantica delle tracce	28
3.5	Calcolo del least fixpoint	29
4	Analisi dataflow	31
4.1	CFGs	31
4.2	Analisi di dataflow	32
4.2.1	Available expressions	33
4.2.2	Altre analisi di dataflow	35
4.3	I reticoli nelle analisi di dataflow	36

4.4	Analisi distributive	37
4.4.1	Constant propagation	38
5	Introduzione a LiSA e architettura di un analizzatore statico	41
5.1	Panoramica di un analizzatore statico	41
5.2	LiSA	42
5.2.1	Overview	42
5.3	Sintassi vs Semantica	44
5.4	Implementazione dell'analisi dataflow	45
6	Il significato dell'approssimazione	48
6.1	Intuizione e necessità dell'approssimazione	48
6.2	Astrazione	48
6.2.1	Oggetti	48
6.2.2	Proprietà e reticolo delle proprietà	49
6.3	Direzione dell'astrazione	50
6.3.1	Approssimazione dal basso	50
6.3.2	Approssimazione dall'alto	51
6.4	Esempi di domini Astratti	52
6.4.1	Dominio dei segni	52
6.5	Intervalli	53
6.6	Ottogoni	53
6.7	Poliedri	53
6.8	Approssimazione delle computazioni	53
6.8.1	Small-step transition semantics	54
6.8.2	Computing on sets (Calcolo su insiemi)	55
6.8.3	Computing by fixpoint (Calcolo tramite punto fisso)	55
6.8.4	Collecting semantics (Semantica collettiva)	57
6.8.5	Computing on properties (Calcolo su proprietà)	58
6.9	Gerarchia delle semantiche come astrazioni	59
7	Galois connections	62
7.1	Dominio concreto	62
7.1.1	Dominio Astratto	63
7.2	Intuizione sulla soundness dei domini astratti	63
7.3	Monotonia	64
7.4	Galois Connection	64
7.5	Galois Insertion (GI)	65
7.6	Best abstraction	65
7.6.1	Problema della best abstraction	66
7.7	Estensione agli stati	66
8	Introduzione all'interpretazione astratta	67
8.1	Semantica astratta sui CFG	67
8.2	Sistemi di Equazioni e punto Fisso	68
8.3	Approssimazione del punto fisso	68
8.3.1	Soundness	69

8.4	Teoremi di trasferimento del punto fisso	70
9	Ottimizzazione dell'Analisi: l'operatore split	71
9.1	Filter operator e problemi associati	71
9.2	Split operator	72
9.3	Implementazione e Integrazione	73
9.3.1	Modifiche al CFG	74
9.4	Valutazione Sperimentale	74
10	Ottimizzazione dell'Analisi Statica: Oracoli per Variabili Non Vincolate	76
10.1	Contesto e abstract compilation	76
10.2	Likely unconstrained variables	76
10.3	La Pipeline di propagazione LUV	77
10.3.1	Oracoli e euristiche	77
10.3.2	Strategie di Propagazione	79
10.4	Trasformazione del programma (Havoc)	79
10.5	Valutazione sperimentale	79
11	Interpretazione Astratta in LiSA	80
11.1	Cos'è LiSA	80
11.2	Architettura di LiSA	80
11.3	Calcolo del punto fisso sul CFG	81
11.4	Semantica degli statement	82
11.5	Gestione della memoria e stato astratto	83
11.5.1	Gestione della memoria dinamica	83
11.5.2	Lo stato astratto	84
11.6	Analisi interprocedurale	85
11.7	Domini astratti: relazionali e non-relazionali	86
11.7.1	Domini non-relazionali	86
11.7.2	Domini relazionali	86
12	Domini numerici e interpretazione astratta	88
12.1	Il problema generale	88
12.2	Esempi di problemi di verifica	88
12.3	Metodi formali di verifica dei programmi	89
12.4	Interpretazione astratta	89
12.5	Domini astratti numerici	90
12.6	Un assaggio di semantica concreta e astratta	90
13	Widening	92
13.1	Definizione dell'operatore di widening	92
13.2	Strategie di applicazione	93
14	Narrowing	94
14.1	Fasi dell'Analisi	94

Disclaimer

Nota

I seguenti appunti sono stati generati a partire dal materiale didattico e dalle note ufficiali del corso “Analisi statica e verifica del software”. Il contenuto è stato riorganizzato, formattato e parzialmente rielaborato con l’ausilio di AI al fine di creare un documento coeso e ottimizzato per lo studio.

Si raccomanda di fare sempre riferimento al materiale originale fornito dal docente per garantire la massima accuratezza.

1 Introduzione

1.1 Cos'è l'affidabilità del software?

Definizione 1.1: Affidabilità del Software (IEEE 610.121990)

“The ability of a system or component to perform its required functions under stated conditions for a specified period of time”

Ovvero:

“La capacità di un sistema o componente di eseguire le funzioni richieste in condizioni specificate per un periodo di tempo specificato”

Definizione 1.2: Gestione dell’Affidabilità del Software (IEEE 982.11988)

“The process of optimizing the reliability of software through a program that emphasizes software error prevention, fault detection and removal, and the use of measurements to maximize reliability in light of project constraints such as resources, schedule and performance”

Ovvero:

“Il processo di ottimizzazione dell’affidabilità del software attraverso un programma che enfatizza la prevenzione degli errori software, il rilevamento e la rimozione dei guasti, e l’uso di misurazioni per massimizzare l’affidabilità alla luce dei vincoli del progetto come risorse, tempi e prestazioni”

Sfruttando le definizioni Definizione 1.1 e Definizione 1.2, è possibile riassumere che l’affidabilità del software consiste in 3 attività:

- Prevenzione degli errori.
- Rilevamento e rimozione dei guasti.
- Valutazione per massimizzare l’affidabilità, in particolare valutazioni a supporto delle prime due attività.

1.2 Perché l’affidabilità del software è importante?

L’importanza dell’affidabilità del software deriva dalle enormi conseguenze economiche e operative che i bug possono causare.

Alcune stime indicano che:

- I bug nei software costano circa 60 miliardi di dollari all’anno negli Stati Uniti.
- L’economia mondiale perde circa 250 miliardi di dollari all’anno a causa di qualsiasi tipo di attacco palese (overt attack).
- I difetti nel software rendono la programmazione molto dolorosa.

Alcuni esempi di fallimenti dovuti a bug software includono:

- Il disastro del volo 501 di Ariane 5 nel 1996, che è esploso dopo 37 secondi dal decollo a causa di un errore di overflow durante la conversione da un float 64 bit ad un intero 16 bit. Questo evento ha causato una perdita di circa 370.000.000 dollari.
- Il bug del Pentium FDIV nel 1994, che ha causato errori di calcolo nelle divisioni in virgola mobile. Questo bug ha portato a una perdita di circa 475.000.000 dollari.
- L'aggiornamento difettoso di CrowdStrike nel 2024, che ha causato più di 5000 voli cancellati. Questo evento ha impattato su banche, governi e infrastrutture critiche. Le perdite economiche sono state stimate a circa 5.400.000.000 dollari.

La lista di esempi potrebbe continuare, ma l'importante è comprendere che l'affidabilità del software è cruciale per evitare perdite economiche significative e garantire il corretto funzionamento dei sistemi.

1.3 Validazione e Verifica

La validazione e la verifica sono due processi distinti ma complementari nell'ambito dello sviluppo del software, entrambi mirati a garantire che il software soddisfi i requisiti e funzioni correttamente.

La **validazione** si concentra sulla domanda: “Stiamo costruendo il prodotto giusto?”. Essa verifica che il software soddisfi le esigenze e le aspettative degli utenti finali. La **verifica**, d'altra parte, si concentra sulla domanda: “Stiamo costruendo il prodotto nel modo giusto?”. Essa assicura che il software sia sviluppato correttamente secondo le specifiche tecniche e i requisiti di progetto.

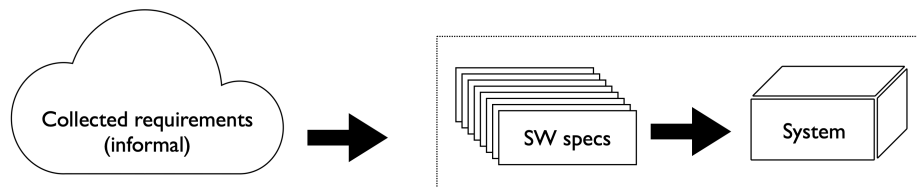


Figura 1: L'immagine illustra la differenza tra validazione e verifica.

Esempio 1.1: Risposta dell'ascensore

Un esempio pratico di validazione e verifica può essere trovato nel comportamento di un sistema di risposta dell'ascensore.

Una specifica validabile ma non verificabile potrebbe essere: “Se un utente preme il pulsante di richiesta a un piano i , un ascensore disponibile deve arrivare al piano i entro un tempo ragionevole”.

Una specifica verificabile è: “Se un utente preme il pulsante di richiesta a un piano i , un ascensore disponibile deve arrivare al piano i entro 30 minuti”.

1.4 Verifica del Software

Lo scopo della verifica del software è verificare alcune affermazioni di correttezza riguardante un programma:

- La **correttezza funzionale**, ovvero che il programma soddisfi i requisiti funzionali specificati.
- L'**assenza di errori non funzionali** (non cosa fa il programma, ma come lo fa), come ad esempio: soddisfacimento di vincoli spaziali e/o temporali, assenza di errori a runtime, soddisfacimento di vincoli di sicurezza, ecc.

Bad news: theres no silver bullet

Non esiste un metodo di verifica che sia contemporaneamente:

- **Automatico**: non richiede interazione umana.
- **Potente**: è in grado di dimostrare proprietà non banali (*non-trivial properties*).
- **Corretto (Sound)**: non dimostra mai una proprietà se questa non è vera.
- **Completo (Complete)**: dimostra sempre la proprietà se essa è vera e non fallisce mai nel dichiarare la correttezza di un programma corretto.

Il **teorema di Rice** afferma che tutte le proprietà non banali del comportamento di un programma scritto in un linguaggio di programmazione Turing-completo sono indecidibili. L'**ind decidibilità** di una proprietà implica che non esiste alcun metodo di verifica automatico che sia allo stesso tempo **corretto** e **completo**.

2 Background matematico

Per poter discutere in modo rigoroso di tecniche di verifica del software, è necessario introdurre alcuni concetti matematici di base relativi alla teoria della computazione e alla logica formale.

2.1 Set notation

Definizione 2.1: Insieme

Un insieme (set) è una collezione di oggetti ben definiti e distinti. La collezione stessa è considerata un oggetto a sé stante.

Dato un insieme S è possibile dire che:

- $s \in S$ significa che l'elemento s appartiene all'insieme S .
- $S_1 \subseteq S_2 \triangleq \forall s \in S_1 \Rightarrow s \in S_2$ significa che l'insieme S_1 è un sottoinsieme di S_2 se ogni elemento di S_1 appartiene anche a S_2 .
- $S_1 \cup S_2 = \{s \mid s \in S_1 \vee s \in S_2\}$ è l'unione di due insiemi, ovvero l'insieme di tutti gli elementi che appartengono ad almeno uno dei due insiemi.
- $S_1 \cap S_2 = \{s \mid s \in S_1 \wedge s \in S_2\}$ è l'intersezione di due insiemi, ovvero l'insieme di tutti gli elementi che appartengono ad entrambi gli insiemi.

2.2 Partial order

Definizione 2.2: Ordine parziale

Un ordine parziale (partial order) è una relazione binaria $\sqsubseteq$ su un insieme X che soddisfa le seguenti proprietà:

- Riflessività: $\forall x \in X \Rightarrow x \sqsubseteq x$, indica che ogni elemento è in relazione con se stesso.
- Anti-simmetria: $\forall x, y \in X, (x \sqsubseteq y \wedge y \sqsubseteq x) \Rightarrow x = y$, indica che se un elemento x è in relazione con un altro elemento y e viceversa, allora x e y sono lo stesso elemento.
- Transitività: $\forall x, y, z \in X, (x \sqsubseteq y \wedge y \sqsubseteq z) \Rightarrow x \sqsubseteq z$, indica che se un elemento x è in relazione con un elemento y , e y è in relazione con un elemento z , allora x è in relazione con z .

Esempio 2.1: Esempio informale di ordine parziale su $\mathbb{Z}$

Informalmente è possibile considerare l'insieme dei numeri interi $\mathbb{Z}$ con la relazione “minore o uguale di” ($\leq$) come un esempio di ordine parziale. In questo caso è possibile verificare che:

- Riflessività: Per ogni numero intero i , vale che $i \leq i$.
- Anti-simmetria: Se $i_1 \leq i_2$ e $i_2 \leq i_1$, allora $i_1 = i_2$.

- **Transitività:** Se $i_1 \leq i_2$ e $i_2 \leq i_3$, allora $i_1 \leq i_3$.

2.3 Powerset

Definizione 2.3: Insieme delle parti

Dato un insieme S , definito in Definizione 2.1, l'insieme delle parti (powerset) di S è l'insieme di tutti i sottoinsiemi di S , ed è indicato con $\wp(S)$.

Dato un insieme S con $|S| = n$ elementi è possibile determinare che l'insieme delle parti di S ha $|\wp(S)| = 2^n$ elementi.

Esempio 2.2: Alcuni esempi di powerset

Dato l'insieme vuoto $\emptyset$, il suo powerset è:

$$\wp(\emptyset) = \{\emptyset\}$$

ed ha $2^0 = 1$ elemento.

Dato un insieme $X = \{a, b\}$, il suo powerset è:

$$\wp(X) = \{\emptyset, \{a\}, \{b\}, \{a, b\}\}$$

ed ha $2^2 = 4$ elementi.

Dato l'insieme $\mathbb{Z}$ dei numeri interi, il suo powerset è:

$$\wp(\mathbb{Z}) = \{\emptyset, \{\dots, -2, -1, 0, 1, 2, \dots\}, \{0\}, \{1, 2, 3, \dots\}, \dots\}$$

ed ha un numero infinito di elementi.

Dato un powerset è possibile definire un ordine parziale $\subseteq$ su di esso e verificare che esso soddisfa le proprietà di un ordine parziale:

- **Riflessività:** $\forall X_1 \in \wp(X). X_1 \subseteq X_1$
- **Anti-simmetria:** $\forall X_1, X_2 \in \wp(X). X_1 \subseteq X_2 \wedge X_2 \subseteq X_1 \Rightarrow X_1 = X_2$
- **Transitività:** $\forall X_1, X_2, X_3 \in \wp(X). X_1 \subseteq X_2 \wedge X_2 \subseteq X_3 \Rightarrow X_1 \subseteq X_3$

Esercizio 2.1: Inclusione inversa su powerset

L'inclusione inversa su un powerset $\wp(X)$ è un ordine parziale? Per verificare se $\supseteq$ è un ordine parziale su $\wp(X)$, è necessario verificare se soddisfa le tre proprietà di un ordine parziale.

- **Riflessività:** $\forall X_1 \in \wp(X) \Rightarrow X_1 \supseteq X_1$, indica che ogni insieme è sovrainsieme di se stesso.
- **Anti-simmetria:** $\forall X_1, X_2 \in \wp(X), (X_1 \supseteq X_2 \wedge X_2 \supseteq X_1) \Rightarrow X_1 = X_2$, indica che se ogni elemento di X_1 è anche in X_2 e che ogni elemento di X_2 è anche in X_1 , allora X_1 e

X_2 sono lo stesso insieme.

- **Transitività:** $\forall X_1, X_2, X_3 \in \wp(X), (X_1 \supseteq X_2 \wedge X_2 \supseteq X_3) \Rightarrow X_1 \supseteq X_3$, indica che se X_1 è un sovrainsieme di X_2 , e X_2 è un sovrainsieme di X_3 , allora X_1 è un sovrainsieme di X_3 .

Poiché tutte e tre le proprietà sono soddisfatte, è possibile concludere che l'inclusione inversa $\supseteq$ è un ordine parziale su il powerset $\wp(X)$.

2.4 Poset

Definizione 2.4: Poset

Un insieme parzialmente ordinato (partially ordered set, poset) è una coppia $\langle X, \sqsubseteq \rangle$ dove X è un insieme (Definizione 2.1) e $\sqsubseteq$ è una relazione di ordine parziale (Definizione 2.2) su X .

Nota 2.1: Esempio di poset

L'esercizio ?? ha mostrato che l'inclusione inversa su un powerset $\wp(X)$ è un ordine parziale. Quindi $\langle X, \supseteq \rangle$ è un poset.

2.5 Diagramma di Hasse

Il diagramma di Hasse è una rappresentazione grafica di un poset.

Sia $X = \{x, y, z\}$ un insieme (Definizione 2.1), ed un poset $\langle X, \sqsubseteq \rangle$ su di esso è possibile rappresentarlo tramite un diagramma di Hasse dove una linea che connette x e y indica che:

- $x \sqsubseteq y$
- $\nexists z \in X$ tale che $x \sqsubseteq z \sqsubseteq y$

È possibile trovare la presenza di più livelli nel diagramma di Hasse, dove un elemento x è posizionato più in alto di un elemento y se x è “immediatamente maggiore” di y .

Nota 2.2: Poset inverso

Un poset inverso è un poset ottenuto invertendo la relazione di ordine di un poset originale. Graficamente, questo si traduce nel riflettere il diagramma di Hasse rispetto ad un asse orizzontale, ovvero ruotando di 180 gradi il diagramma.

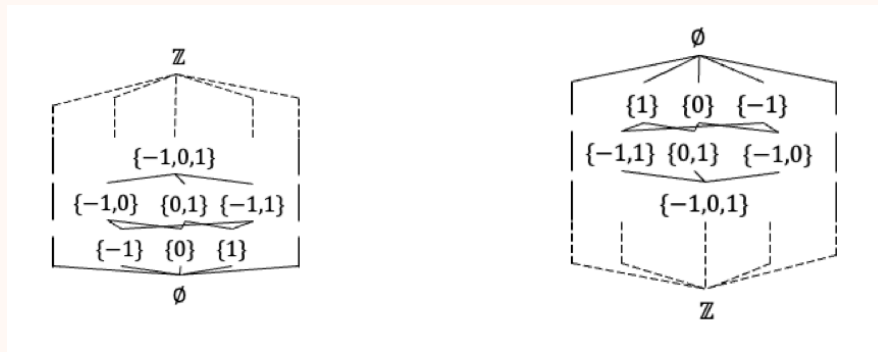


Figura 2: L'immagine illustra un esempio di diagramma di Hasse per un poset e il suo poset inverso.

2.6 Upper e lower bounds

Definizione 2.5: Upper bound and least upper bound

Dato un poset $\langle X, \sqsubseteq \rangle$ e un sottoinsieme $Y \subseteq X$ è possibile dire che:

1. $y \in X$ è un upper bound di Y se $\forall y' \in Y, y' \sqsubseteq y$, ovvero se ogni elemento dell'insieme Y è minore o uguale a y .
2. y è l'upper bound più piccolo tra tutti gli upper bound di Y , ovvero $\forall y' \in UB(Y). y \sqsubseteq y'$, dove $UB(Y)$ è l'insieme di tutti gli upper bound di Y .

Definizione 2.6: Lower bound e greatest lower bound

Dato un poset $\langle X, \sqsubseteq \rangle$ e un sottoinsieme $Y \subseteq X$ è possibile dire che:

1. $y \in X$ è un lowerbound di Y se $\forall y' \in Y, y \sqsubseteq y'$, ovvero se ogni elemento dell'insieme Y è maggiore o uguale a y .
2. y è il lowerbound più grande tra tutti i lowerbound di Y , ovvero $\forall y' \in LB(Y). y' \sqsubseteq y$, dove $LB(Y)$ è l'insieme di tutti i lowerbound di Y .

Esercizio 2.2: Lub e glb su powerset

Dato il Poset $\langle \wp(X), \subseteq \rangle$, e siano $S_1, S_2 \in \wp(X)$:

1. $S_1 \cup S_2$ è l'estremo superiore (lub) di $\{S_1, S_2\}$?
2. $S_1 \cap S_2$ è l'estremo inferiore (glb) di $\{S_1, S_2\}$?

Soluzione esercizio 1

$S_1 \cup S_2$ è l'estremo superiore (lub) di $\{S_1, S_2\}$?

Sia $L = S_1 \cup S_2$, che per essere lub deve soddisfare le condizioni elencate in Definizione 2.5 punto 1 e 2.

Punto 1:

$S_1 \subseteq L$ e $S_2 \subseteq L$.

Punto 2:

Sia U un qualsiasi insieme tale che $S_1 \subseteq U$ e $S_2 \subseteq U$. Allora U deve per contenere gli elementi di S_1 e S_2 deve essere proprio $(S_1 \cup S_2) \subseteq U$.

Soluzione esercizio 2

$S_1 \cap S_2$ è l'estremo inferiore (glb) di $\{S_1, S_2\}$?

2.7 Supremum e Infimum**Definizione 2.7: Supremum**

Dato un poset $\langle X, \sqsubseteq \rangle$ e un sottoinsieme $S \subseteq X$ il supremum (o least upper bound) di S è l'elemento più piccolo tra tutti gli upper bound di S .

Definizione 2.8: Infimum

Dato un poset $\langle X, \sqsubseteq \rangle$ e un sottoinsieme $S \subseteq X$ l'infimum (o greatest lower bound) di S è l'elemento più grande tra tutti i lower bound di S .

2.8 Proprietà di lub e glb**Proposizione 2.1: Proprietà di lub e glb**

Dato un poset $\langle X, \sqsubseteq \rangle$ è possibile definire:

- $\sqcup$ come il lub su X .
- $\sqcap$ come il glb su X .
- se $\sqcup$ e $\sqcap$ esistono sono unici.
- $\sqcup X$ esiste se e solo se X ha un elemento $\top$ ($\sqcup X = \top$).
- $\sqcap X$ esiste se e solo se X ha un elemento $\perp$ ($\sqcap X = \perp$).

2.9 Reticolo

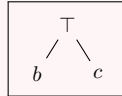
Definizione 2.9: Reticolo

Dato un poset $\langle X, \sqsubseteq \rangle$, esso è un reticolo (lattice) se soddisfa entrambe le seguenti proprietà:

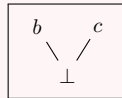
- $\forall x, y \in X, \exists x \sqcup y$, ovvero per ogni coppia di elementi qualsiasi di X deve esistere il loro lub. Un poset che soddisfa questa proprietà è detto “join semi lattice”.
- $\forall x, y \in X, \exists x \sqcap y$, ovvero per ogni coppia di elementi qualsiasi di X deve esistere il loro glb. Un poset che soddisfa questa proprietà è detto “meet semi lattice”.

In un reticolo è presente la relazione d'ordine parziale $\sqsubseteq$ su due elementi $x, y \in X$, ovvero $x \sqsubseteq y$ se e solo se:

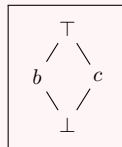
- $x \sqcup y = y$
- $x \sqcap y = x$



Join semi-lattice



Meet semi-lattice



Lattice

Figura 3: Rappresentazione grafica di Join semi-lattice, Meet semi-lattice e Lattice.

2.10 Set lattice

Definizione 2.10: Set lattice

Le operazioni insiemistiche definiscono una struttura di reticolo. Dato il poset $\langle \wp(X), \subseteq \rangle$, esso forma un reticolo rispetto alle operazioni di unione ($\cup$) e intersezione ($\cap$), denotato come:

$$\langle \wp(X), \subseteq, \cup, \cap \rangle$$

Se l'insieme X è finito, allora il reticolo ha altezza finita.

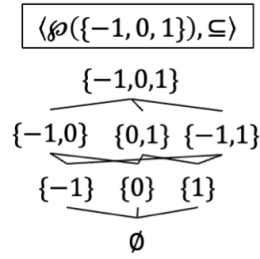


Figura 4: L'immagine illustra un esempio di reticolo su un poset finito.

Nota 2.3: Top e Bottom nei reticoli

Un reticolo non possiede necessariamente un elemento Top ($\top$) o un elemento Bottom ($\perp$). Ad esempio, il reticolo $\langle \mathbb{Z}, \leq, \max, \min \rangle$ non ha né un elemento massimo né un elemento minimo, poiché l'insieme dei numeri interi è illimitato in entrambe le direzioni.

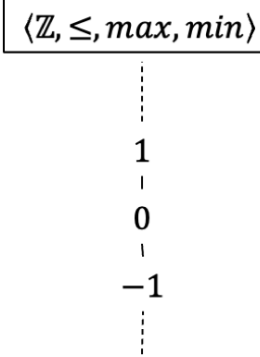


Figura 5: L'immagine illustra il reticolo su $\mathbb{Z}$, privo di elementi top e bottom.

2.11 Reticolo completo

Definizione 2.11: Reticolo completo

Un reticolo $\langle X, \sqsubseteq, \sqcup, \sqcap \rangle$ (Definizione 2.9) è detto completo se:

- Ogni sottoinsieme $Y \subseteq X$ ha un lub (Definizione 2.5) in X .
- Ogni sottoinsieme $Y \subseteq X$ ha un glb (Definizione 2.6) in X .

quindi si denota come:

$$\langle X, \sqsubseteq, \perp, \top, \sqcup, \sqcap \rangle$$

Nota 2.4: Lub del sottoinsieme vuoto

Se ogni sottoinsieme $Y \subseteq X$ ha un lub in X , allora anche il sottoinsieme vuoto $\emptyset$ deve avere un lub in X . In questo caso il lub del sottoinsieme vuoto è l'elemento bottom $\perp$ del reticolo.

Esempio 2.3: Esempi di reticolo completo e non completo

Considerando l'insieme dei numeri interi $\mathbb{Z}$ con la relazione “minore o uguale di” ($\leq$), si può osservare che

$$\langle \mathbb{Z}, \leq, \min, \max \rangle$$

è un reticolo ma non è completo. Infatti, non esiste un elemento top ($\top$) in $\mathbb{Z}$, poiché non esiste un numero intero massimo.

Se all'insieme dei numeri interi si aggiungono gli elementi $+\infty$ e $-\infty$, si ottiene l'insieme esteso $\mathbb{Z} \cup \{+\infty, -\infty\}$. In questo caso, il reticolo

$$\langle \mathbb{Z} \cup \{+\infty, -\infty\}, \leq, \min, \max \rangle$$

diventa un reticolo completo, poiché ora esistono sia un elemento top ($+\infty$) che un elemento bottom ($-\infty$).

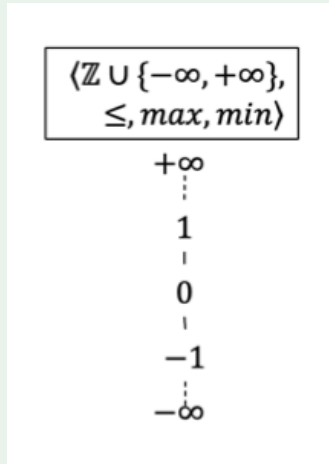


Figura 6: L'immagine illustra il reticolo su $\mathbb{Z} \cup \{+\infty, -\infty\}$.

Proposizione 2.2: Proprietà di un reticolo completo

Un reticolo completo possiede le seguenti proprietà:

- reticolo completo $\langle X, \sqsubseteq, \sqcup, \sqcap \rangle$ ha un elemento top $\top$ e un elemento bottom $\perp$ e si denota come

$$\langle X, \sqsubseteq, \perp, \top, \sqcup, \sqcap \rangle$$

- I reticoli finiti sono completi.
- $\sqcup$ induce $\sqcap : \sqcap S = \sqcup \{y \mid \forall x \in S. y \sqsubseteq x\}$
- $\sqcap$ induce $\sqcup : \sqcup S = \sqcap \{y \mid \forall x \in S. x \sqsubseteq y\}$

2.12 Relations**Definizione 2.12: Prodotto cartesiano**

Dati due insiemi (Definizione 2.1) X e Y , il loro prodotto cartesiano $X \times Y$ è l'insieme di tutte le possibili coppie ordinate (x, y) dove il primo elemento appartiene a X e il secondo a Y .

$$X \times Y = \{(x, y) \mid x \in X \wedge y \in Y\}$$

Definizione 2.13: Relazione

Una relazione R tra due insiemi X e Y è definita come un sottoinsieme del loro prodotto cartesiano (Definizione 2.12):

$$R \subseteq X \times Y$$

Nota 2.5: Notazione per le relazioni

Dato una relazione $R \subseteq X \times Y$, è possibile indicare che un elemento $x \in X$ è in relazione con un elemento $y \in Y$ tramite due notazioni:

- $(x, y) \in R$, ovvero tramite relazione insiemistica.
- $x R y$, ovvero tramite notazione infissa.

Nota 2.6: Ordine parziale come relazione

Un ordine parziale $\sqsubseteq$ (Definizione 2.2) è un caso particolare di relazione definita sullo stesso insieme X . Formalmente, $\sqsubseteq$ è un sottoinsieme del prodotto cartesiano $X \times X$ ($\sqsubseteq \subseteq X \times X$). A differenza di una relazione generica, l'ordine parziale deve soddisfare le proprietà di riflessività, anti-simmetria e transitività (Definizione 2.2).

2.13 Functions (or maps)

Le funzioni (functions), sono un tipo particolare di relazione, e sono fondamentali per riuscire a modellare gli stati di un programma.

Definizione 2.14: Funzione

Una funzione f tra due insiemi X e Y è un particolare tipo di relazione $R \subseteq X \times Y$ che soddisfa la proprietà di unicità dell'immagine.

Formalmente, per ogni elemento x nel dominio, esiste al massimo un elemento y nel codominio tale che $(x, y) \in f$:

$$\forall (x, y) \in f, \nexists (x', y') \in f. (x = x' \wedge y \neq y')$$

Una funzione è definita al massimo una volta su un dato input.

Esempio 2.4: Cerchio su un piano cartesiano

Un cerchio su un piano cartesiano non rappresenta una funzione, poiché un singolo valore di x può essere associato a due valori distinti di y (uno sopra l'asse x e uno sotto). Questo viola la proprietà di unicità dell'immagine definita per le funzioni.

Nota 2.7: Non tutte le relazioni sono funzioni

Un ordine parziale (Definizione 2.2) non è generalmente una funzione, poiché un elemento può essere minore o uguale a molti altri elementi (uno-a-molti).

Definizione 2.15: Notazione delle funzioni

Dato $f : X \rightarrow Y$, dove X è il dominio e Y è il codominio:

- Notazione estensionale: Una funzione definita su punti specifici si scrive come:

$$f = [x_0 \mapsto y_0, x_1 \mapsto y_1, \dots, x_n \mapsto y_n]$$

che è equivalente all'insieme di coppie $\{(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)\}$.

- Lambda notazione: Si utilizza $\lambda x. espressione$ per definire una funzione anonima. Ad esempio $\lambda x. f(x) \equiv f$.

2.13.1 Monotonia, Embedding e Isomorfismo

Definizione 2.16: Funzioni monotone, embedding ordinato e isomorfe

Dati due posets $\langle X, \sqsubseteq_X \rangle$ e $\langle Y, \sqsubseteq_Y \rangle$, e una funzione $f : X \rightarrow Y$, è possibile dire che:

- La funzione f è monotona se $x_1 \sqsubseteq_X x_2 \Rightarrow f(x_1) \sqsubseteq_Y f(x_2)$.
- La funzione f è un embedding ordinato se $x_1 \sqsubseteq_X x_2 \iff f(x_1) \sqsubseteq_Y f(x_2)$.

- La funzione f è un isomorfa se è un embedding ordinato ed è suriettiva, ovvero: $\forall y \in Y, \exists x \in X. f(x) = y$.

Esempio 2.5: Esempio di funzione monotona, embedding ordinato e suriettiva

Data $f : \langle \mathbb{Z}, \leq \rangle \rightarrow \langle \mathbb{Z}, \leq \rangle$, definita come $f(x) = x + 1$, è possibile osservare che:

- La funzione è monotona.
- La funzione è un embedding ordinato.
- La funzione è suriettiva.

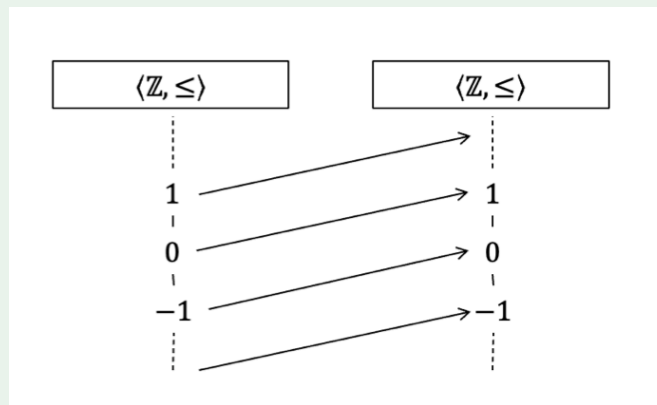


Figura 7: L'immagine illustra la funzione $f(x) = x + 1$ come monotona, embedding ordinato e suriettiva.

Esempio 2.6: Esempio di funzione monotona ma che non è un embedding

Data $f : \langle \wp(\mathbb{Z}), \subseteq \rangle \rightarrow \langle \mathbb{Z}, \leq \rangle$, definita come $f(X) = \max(X)$, è possibile osservare che:

- La funzione è monotona.
- La funzione non è un embedding.

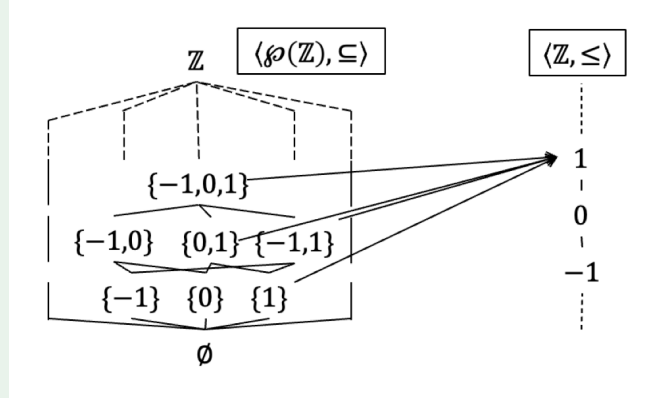


Figura 8: L'immagine illustra la funzione $f(X) = \max(X)$ come monotona ma non un embedding.

2.13.2 Funzioni che preservano join e meet

Dati due reticoli $\langle X, \sqsubseteq_X, \sqcup_X, \sqcap_X \rangle$ e $\langle Y, \sqsubseteq_Y, \sqcup_Y, \sqcap_Y \rangle$, e una funzione $f: X \rightarrow Y$, è possibile dire che:

- La funzione f preserva i join se $f(x_1 \sqcup_X x_2) = f(x_1) \sqcup_Y f(x_2)$.
- La funzione f preserva i meet se $f(x_1 \sqcap_X x_2) = f(x_1) \sqcap_Y f(x_2)$.

Nota 2.8: Mantenimento della Monotonia

Una funzione che preserva i join o i meet è monotona, ma una funzione che è monotona non necessariamente preserva i join o i meet.

2.14 Chains

Definizione 2.17: Catena

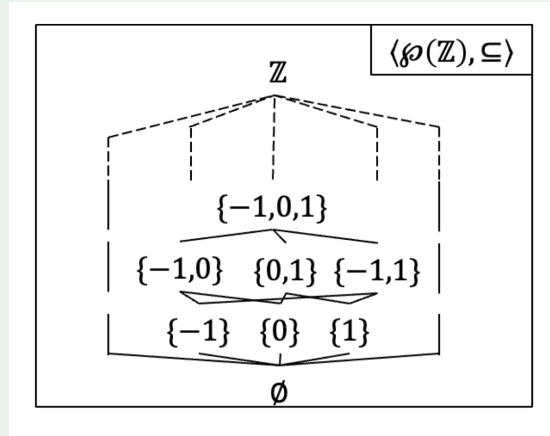
Dato un poset $\langle X, \sqsubseteq \rangle$, una catena (chain) S se tutti gli elementi contenuti sono completamente ordinati, ovvero:

$$\forall x, y \in S. x \sqsubseteq y \vee y \sqsubseteq x$$

Esempio 2.7: Esempio di catena

Dato un poset $\langle \wp(\mathbb{Z}), \sqsubseteq \rangle$ con reticolo rappresentato nell'immagine sottostante (9), esempi di catene sono:

- $\{\emptyset\}$
- $\{\emptyset, \{0, 1\}\}$
- $\{\{0\}, \{-1, 0, 1\}\}$

Figura 9: L'immagine illustra un reticolo su $\wp(\mathbb{Z})$.

2.14.1 Ascending Chain Condition (ACC)

Definizione 2.18: Catena Ascendente

Una catena ascendente è una sequenza di elementi $(l_n)_{n \in \mathbb{N}}$ indicizzata dai numeri naturali tali che:

$$i \leq j \implies l_i \subseteq l_j$$

Ovvero $l_0 \subseteq l_1 \subseteq l_2 \subseteq \dots$

Definizione 2.19: Ascending Chain Condition (ACC)

Un poset soddisfa la Ascending Chain Condition (ACC) se ogni catena ascendente infinita si stabilizza (non è strettamente crescente per sempre).

Matematicamente, una catena (l_n) si stabilizza se:

$$\exists k \geq 0. \forall j \geq k. l_k = l_j$$

Questo significa che, dopo un certo indice k , tutti gli elementi successivi sono uguali a l_k (il punto fisso della sequenza).

Esempio 2.8: Esempio di catena ascendente infinita non è strettamente crescente

Un esempio di catena ascendente infinita non strettamente crescente è la seguente:

$$\exists k \geq 0. \forall j \geq k. l_k = l_j$$

Perché dopo un certo numero di passi k , tutti gli elementi successivi della catena sono uguali a l_k . Ovvero la catena si stabilizza.

2.15 Mappe continue

Definizione 2.20: Mappa continua

Dati due reticoli (o poset) $\langle X, \sqsubseteq_X, \sqcup_X, \sqcap_X \rangle$ e $\langle Y, \sqsubseteq_Y, \sqcup_Y, \sqcap_Y \rangle$ e una funzione $f : X \rightarrow Y$ (Definizione 2.14), essa è detta continua se per ogni catena $C \subseteq X$ (Definizione 2.17) tale che $\sqcup_X C$ esiste, valgono le seguenti condizioni:

- $\sqcup_Y f(C)$ esiste;
- $f(\sqcup_X C) = \sqcup_Y f(C)$.

Nota 2.9: Utilità delle mappe continue

Le mappe continue sono fondamentali nell'analisi statica perché permettono di applicare il Teorema di Kleene per calcolare i punti fissi (fixpoints) tramite iterazione.

2.16 Fixpoint

L'analisi statica riduce spesso il problema della verifica di un programma alla ricerca della soluzione di un sistema di equazioni ricorsive. La soluzione di tali equazioni corrisponde al concetto matematico di punto fisso.

Definizione 2.21: Punto Fisso

Dato un insieme X e una funzione $f : X \rightarrow X$:

- Un elemento $x \in X$ è un punto fisso di f se $f(x) = x$.
- L'insieme di tutti i punti fissi di f è denotato come $Fix(f) = \{x \mid f(x) = x\}$.

In un poset $\langle X, \sqsubseteq \rangle$, sono interessanti due tipi particolari di punti fissi:

- Least Fixpoint (lfp), ovvero il punto fisso più piccolo.

$$\forall y \in Fix(f). x \sqsubseteq y$$

- Greatest Fixpoint (gfp), ovvero il punto fisso più grande.

$$\forall y \in Fix(f). x \sqsupseteq y$$

2.16.1 Iterazioni di una funzione

Per calcolare un punto fisso, spesso si ricorre all'applicazione ripetuta della funzione.

Definizione 2.22: Iterazioni

Le iterate di una funzione $f : X \rightarrow X$ a partire da un elemento x sono definite per induzione come:

- $f^0(x) = x$
- $f^{n+1}(x) = f(f^n(x))$

Nota 2.10: Stabilizzazione

Se l'insieme X è finito, la sequenza delle iterazioni $f^0, f^1, \dots$ deve necessariamente stabilizzarsi (raggiungere un punto fisso o entrare in un ciclo) in al massimo $|X|$ passi. Formalmente:

$$\forall k \geq |X|. \exists n < |X|. f^k(x) = f^n(x)$$

Se X è infinito, la sequenza potrebbe non terminare mai.

2.17 Teoremi di Punto Fisso

Esistono due teoremi principali che garantiscono l'esistenza (e talvolta la costruttibilità) dei punti fissi. La scelta del teorema dipende dalle proprietà della funzione f (Definizione 2.16)

Teorema 2.1: Teorema di Knaster-Tarski

Sia $\langle X, \sqsubseteq, \sqcup, \sqcap \rangle$ un reticolo e sia $f : X \rightarrow X$ una funzione monotona. Allora:

1. $\langle \text{Fix}(f), \sqsubseteq, \sqcup, \sqcap \rangle$ è un reticolo completo.
2. Il least fixpoint (lfp) è dato dal meet di tutti i “post-fixpoints”, ovvero $\text{lfp}(f) = \sqcap \{l \mid l \sqsupseteq f(l)\}$
3. Il greatest fixpoint (gfp) è dato dal join di tutti i “pre-fixpoints”, ovvero $\text{gfp}(f) = \sqcup \{l \mid l \sqsubseteq f(l)\}$

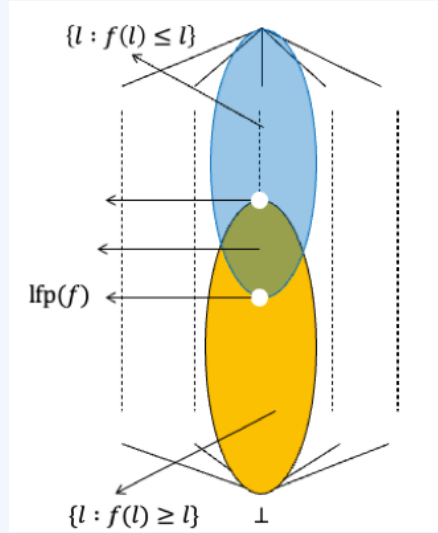


Figura 10: L'immagine illustra un esempio grafico del Teorema di Knaster-Tarski.

Teorema 2.2: Teorema di Kleene

Sia $\langle X, \sqsubseteq, \perp, \top, \sqcup, \sqcap \rangle$ un reticolo completo e sia $f : X \rightarrow X$ una funzione continua (Definizione 2.16). Allora f ha un least fixpoint che può essere calcolato costruttivamente come il limite della catena ascendente che parte dal bottom ($\perp$):

$$lfp(f) = \bigsqcup_{n \geq 0} f^n(\perp)$$

Nota 2.11: Confronto tra i Teoremi

I due teoremi di punto fisso presentano differenze chiave:

- Knaster-Tarski: Richiede solo la monotonia. Questo teorema è più generale ma non fornisce un algoritmo costruttivo (dice solo “esiste ed è uguale all’intersezione di ...”).
- Kleene: Richiede la continuità (condizione più forte). Questo teorema fornisce un algoritmo iterativo $(f(\perp), f(f(\perp)), \dots)$ per calcolare il risultato.

3 Modellazione dei programmi

L'obiettivo dell'analisi statica è certificare che un programma sia sicuro rispetto a determinati errori, come ad esempio la divisione per zero.

3.1 Poset nell'analisi statica

L'idea fondamentale è calcolare i possibili valori che una variabile può assumere in ogni punto del programma, considerando tutte le possibili esecuzioni. Per fare questo è possibile utilizzare i reticoli (Definizione Definizione 2.9).

Esempio 3.1: Esempio di reticolo su un programma

Considerando il programma seguente:

```
1: i = read ();
2 if ( i != 0 )
3   j = 5 / i ;
4 else
5   j = 0;
6 return;
```

e definendo il reticolo dei valori interi come $\langle \wp(\mathbb{Z}), \subseteq, \cup, \cap \rangle$ è possibile calcolare i possibili valori delle variabili i e j in ogni punto del programma.

Program point	Valori
1	$i = \{\}, j = \{\}$
2	$i = \mathbb{Z}, j = \{\}$
3	$i = \mathbb{Z} \setminus \{0\}, j = \{5/i\}$
5	$i = \{0\}, j = \{0\}$
6	$i = \mathbb{Z}, j = \{-5, -2, -1, 0, 1, 2, 5\}$

Per ottenere i valori di j al punto 6, è stato necessario calcolare l'unione dei valori di j provenienti dai punti 3 e 5, utilizzando l'operazione di least upper bound (operazione di join, unione insiemistica in questo caso).

Per ottenere i valori di i al punto 2, è stato necessario considerare che la funzione `read()` può restituire qualsiasi valore intero. Dopo di che tramite un'operazione di "filtro" $filter(i \neq 0)$ è possibile restringere i valori di i per entrare nel ramo vero dell'if. Per il calcolo del punto 3, invece è stato necessario applicare l'operazione di intersezione tra i valori di i al punto 2 e il filtro $filter(x \neq 0)$ per ottenere i valori di i validi per quel ramo.

Ipotizzando che la funzione `read()` possa restituire valori interi in $\{-1, 0, 1\}$, è possibile riscrivere il codice del programma di esempio come:

```
1: i = {-1, 0, 1} ;
2 if ( i != 0 )
```

```

3  j = 5 / i ;
4  else
5  j = 0;
6  return;

```

Di conseguenza i valori delle variabili in ogni punto del programma diventano:

Program point	Valori
1	$i = \{\}, j = \{\}$
2	$i = [-1, 1], j = \{\}$
3	$i = [-1, -1] \cup [1, 1], j = [-5, 5]$
5	$i = [0, 0], j = [0, 0]$
6	$i = [-1, 1], j = [-5, 5]$

3.2 Punti fissi

Quando il programma contiene dei cicli (come **while**), il numero di esecuzioni possibili diventa arbitrario o infinito. Per analizzare queste strutture, si utilizza il concetto di punto fisso (Definizione 2.21). L'analisi procede iterativamente accumulando informazioni (tramite il Join $\cup$) finché lo stato non si stabilizza.

Esempio 3.2: Analisi di un ciclo su dominio infinito (Non terminazione)

Considerando il programma seguente che presenta un ciclo infinito che incrementa un contatore:

```

1: i = 0;
2: while (?)
3:   i++;
4: ...

```

Utilizzando il reticolo $\langle \wp(\mathbb{Z}), \subseteq, \cup, \cap \rangle$, è possibile calcolare i valori di i al punto 2 (l'ingresso del ciclo). Lo stato al punto 2 è definito come l'unione del valore iniziale (da 1) e del valore di ritorno dal ciclo (da 3):

$$Val(2) = \{0\} \cup (Val(3) + 1)$$

Passo passo:

Iterazione	Valori al Punto 2	Valori al Punto 3
0	$\emptyset$	$\emptyset$
1	$\{0\}$	$\emptyset$
2	$\{0\}$	$\{0\}$
3	$\{0\} \cup \{0+1\} = \{0, 1\}$	$\{0\}$
4	$\{0, 1\}$	$\{0, 1\}$
5	$\{0, 1\} \cup \{1+1\} = \{0, 1, 2\}$	$\{0, 1\}$
...	...	...

Si nota che l'insieme continua a crescere $(\{0, 1, 2, 3, \dots\})$ e, poiché il reticolo $\wp(\mathbb{Z})$ ha altezza infinita, l'algoritmo non raggiunge mai un punto fisso in tempo finito.

3.3 IMP

IMP è un linguaggio imperativo minimale che supporta assegnamenti, sequenze, condizionali e cicli.

3.3.1 Sintassi

La sintassi del linguaggio è definita dalle seguenti produzioni grammaticali:

- **Espressioni aritmetiche (e):**

$$e ::= x \mid n \mid e_1 \text{ op}_a e_2$$

dove x è una variabile, $n \in \mathbb{Z}$ è un intero, e $\text{op}_a \in \{+, -, *, \div\}$.

- **Espressioni booleane (b):**

$$b ::= \text{true} \mid \text{false} \mid \neg b \mid b_1 \text{ op}_b b_2 \mid e_1 \text{ op}_c e_2$$

dove $\text{op}_b \in \{\&\&, ||\}$ (logici) e $\text{op}_c \in \{==, <, >, \leq, \geq\}$ (confronto).

- **Statement (s):**

$$s ::= x := e; \mid \text{skip} \mid s_1 s_2 \mid \text{if } b \text{ then } s_1 \text{ else } s_2 \mid \text{while } b \text{ do } s_1$$

3.3.2 Semantica concreta

3.4 Semantica delle tracce

Per analizzare il comportamento di un programma, è necessario definire cosa significa “eseguirlo”. La semantica di traccia modella l'esecuzione come una sequenza (traiettoria) di stati nel tempo.

Definizione 3.1: Traccia

Una traccia $\tau \in X^\infty$ è una sequenza di stati che rappresenta una singola evoluzione del programma. Le tracce possono essere:

- **Finite:** L'esecuzione termina in uno stato finale.
- **Infinite:** L'esecuzione non termina (es. ciclo infinito).
- **Parziali:** Rappresentano un prefisso dell'esecuzione fino a un certo punto intermedio.

Definizione 3.2: Semantica delle tracce

È possibile modellare la semantica concreta, ovvero l'insieme di tutte le possibili esecuzioni, utilizzando un reticolo definito come:

$$\langle \wp(X^\infty), \subseteq, \cup, \cap \rangle$$

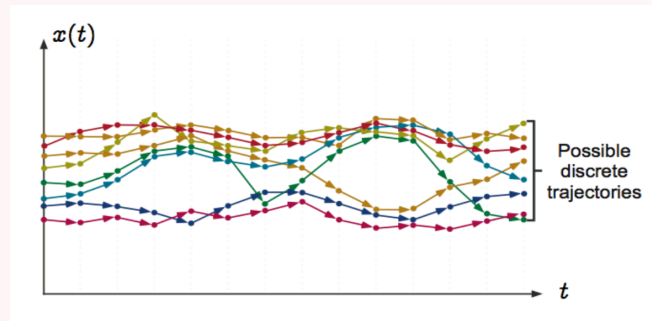


Figura 11: L'immagine illustra l'insieme delle tracce di un programma.

Nota 3.1: Calcolo della semantica

La semantica del programma viene calcolata come il least fixpoint (lfp) di una funzione che costruisce le tracce passo dopo passo:

1. Si parte dalle tracce di lunghezza 1 (stati iniziali).
2. Si applica iterativamente la funzione di transizione per estendere le tracce parziali.
3. Si uniscono ($\cup$) i risultati fino a raggiungere il punto fisso.

Questo processo cattura sia le esecuzioni che terminano, sia quelle che divergono (loop infiniti).

3.5 Calcolo del least fixpoint

Il calcolo della semantica di un programma P si riduce alla ricerca del least fixpoint (lfp) della sua funzione semantica F_P .

Proposizione 3.1: Procedura Iterativa

Il least fixpoint si calcola iterando la funzione F_P a partire dall'elemento *bottom* ($\perp$) del reticolo, procedendo dal “basso verso l'alto”:

$$lfp(F_P) = \bigsqcup_{n \in \mathbb{N}} F_P^n(\perp)$$

La sequenza delle iterazioni costruisce progressivamente la semantica:

- Passo 0: $F^0(\perp) = \perp$ (stato iniziale vuoto o non inizializzato).
- Passo 1: $F^1(\perp) = F(\perp)$ (prime esecuzioni parziali, es. stati iniziali).
- Passo k : $F^k(\perp)$ (insieme delle esecuzioni parziali di lunghezza fino a k).

Il processo termina quando si raggiunge un punto fisso, ovvero quando $F^{k+1}(\perp) = F^k(\perp)$.

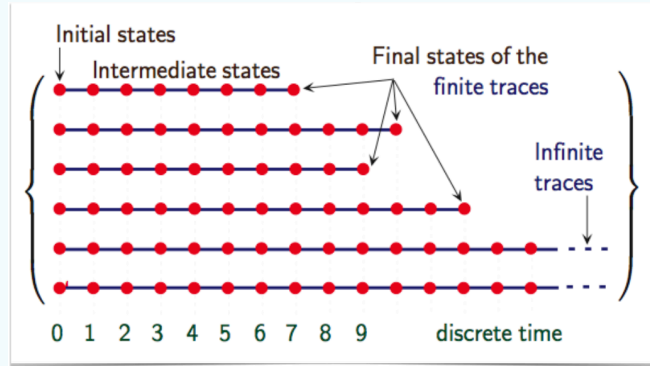


Figura 12: L'immagine illustra l'insieme delle tracce finite e infinite di un programma, con gli stati iniziali, stati intermedi e stati finali.

Nota 3.2: Significato delle iterazioni

In questo modo, ad ogni passo dell'iterazione si ottiene un insieme di esecuzioni parziali. L'unione di tutte queste iterazioni fino al punto fisso fornisce la semantica completa del programma, che include sia le tracce finite che quelle infinite.

4 Analisi dataflow

4.1 CFGs

Nelle analisi di tipo dataflow i programmi vengono rappresentati come control flow graph (CFG), dove i nodi rappresentano gli statement del programma e gli archi rappresentano i possibili flussi di controllo tra di essi.

Esempio 4.1: CFG di un programma semplice

Si consideri il seguente programma:

```
1 y = a - b ;  
2 x = y + b ;  
3 while ( y > a + b )  
4   y = y - x * x ;
```

La sua rappresentazione come CFG è la seguente:

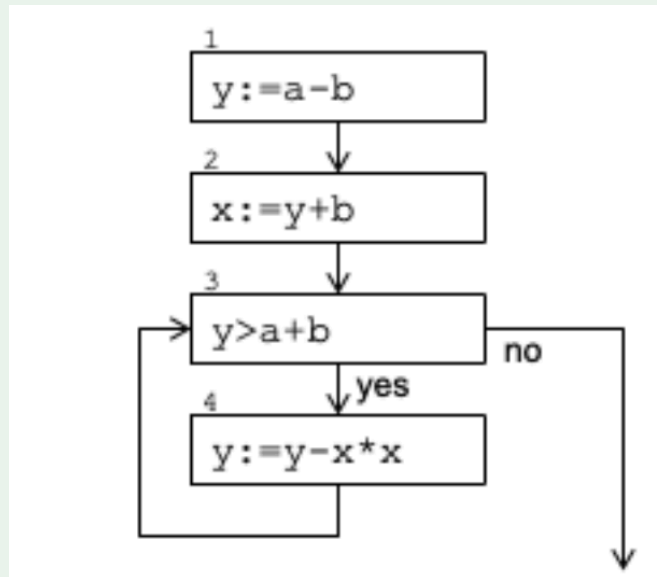


Figura 13: L'immagine illustra il CFG del programma di esempio.

Definizione 4.1: Struttura di un CFG

Dato un programma P , il suo control flow graph (CFG) è formato da:

- Un insieme di nodi (blocchi) identificati da $i \in \mathbb{N}$
- Un insieme di archi rappresentati come coppie $(i_1, i_2) \in \mathbb{N} \times \mathbb{N}$ dove i_1, i_2 sono nodi del CFG.

Dato un CFG, è possibile definire:

- $nodes(G)$ come l'insieme di tutti gli indici dei blocchi in G .
- $stmt(G, i)$ lo statement associato al blocco i in G .
- $edges(G) \subseteq \mathbb{N} \times \mathbb{N}$ come l'insieme di tutti gli archi in G .

4.2 Analisi di dataflow

Le analisi di dataflow sono tecniche tradizionali utilizzate principalmente per le ottimizzazioni nei compilatori. Queste analisi si basano sulla rappresentazione del programma tramite CFG e utilizzano un approccio formale basato su sistemi di equazioni.

Le equazioni vengono costruite definendo le relazioni tra gli stati entranti (*entry*) e uscenti (*exit*) di ogni blocco del CFG.

Nota 4.1: Scopo dell'analisi

La proprietà di interesse è solitamente semplice e orientata a ottimizzare l'esecuzione a runtime (ad esempio, evitando di calcolare due volte la stessa espressione).

Nota 4.2: Classificazione delle analisi di dataflow

Le analisi dataflow classiche possono essere classificate secondo due criteri principali:

- **Direzione del flusso:** indica se l'analisi procede seguendo il flusso di controllo del programma (**forward**) o contro di esso (**backward**).
- **Semantica della proprietà:** indica se l'analisi determina ciò che **può** accadere (**may/possible**) o ciò che **deve** accadere (**must/definite**).

Il tipo di classificazione determina importanti aspetti formali:

- **Forward vs Backward:**
 - Forward: $entry(i)$ dipende da $exit(j)$ dei predecessori
 - Backward: $exit(i)$ dipende da $entry(j)$ dei successori
- **May vs Must:**
 - May (possible): si utilizza l'unione ($\cup$) al join point (basta che la proprietà valga lungo *un* cammino)
 - Must (definite): si utilizza l'intersezione ($\cap$) al join point (la proprietà deve valere lungo *tutti* i cammini)

Queste caratteristiche determinano la struttura del reticolo e l'operatore di join utilizzato nell'analisi.

4.2.1 Available expressions

Definizione 4.2: Available expressions

“For each program point, which expressions must have already been computed, and not later modified, on all paths to the program point”
 Formalmente un'espressione e è disponibile in un punto del programma p se e solo se e è stata calcolata lungo ogni cammino che porta a p e nessuna delle variabili che occorrono in e è stata ridefinita dopo il calcolo di e .

Nota 4.3: Obiettivi dell'analisi

Quest'analisi è un'analisi **forward** e **must** che ha come obiettivi principali:

- Eliminazione di sottoespressioni comuni (Common Subexpression Elimination): evitare di ricalcolare espressioni il cui valore è già disponibile.
- Allocazione ottimale dei registri: memorizzare le espressioni più frequentemente utilizzate in registri temporanei.

Dominio dell'analisi Sia AE l'insieme di tutte le espressioni aritmetiche non banali che compaiono nel programma. Lo stato dell'analisi in ogni punto del programma è un elemento di $\wp(AE)$, ovvero un sottoinsieme delle espressioni aritmetiche.

Definizione 4.3: Funzioni ausiliarie

È possibile definire le seguenti funzioni ausiliarie:

- $AExp : Expr \rightarrow \wp(AE)$ restituisce l'insieme delle espressioni aritmetiche non banali contenute in un'espressione e .
- $AExpB : BExpr \rightarrow \wp(AE)$ restituisce l'insieme delle espressioni aritmetiche non banali contenute in un'espressione booleana b .
- $FV : AE \rightarrow \wp(Var)$ restituisce l'insieme delle variabili libere (free variables) che occorrono in un'espressione a .

Funzioni di trasferimento Per ogni statement del programma, è necessario definire le funzioni *gen* e *kill* che descrivono come lo statement modifica l'insieme delle espressioni disponibili.

Definizione 4.4: Gen e Kill

Considerando un assegnamento $x := e$, dove x è una variabile ed e un'espressione.

- **Killed:** Quando x viene assegnata, tutte le espressioni che contengono x non sono più valide (vengono “uccise”) perché il valore di x è cambiato.

$$kill(x := e) = \{a \in AE \mid x \in FV(a)\}$$

Dove $FV(a)$ è l'insieme delle variabili libere in a .

- **Gen:** Quando x viene assegnata, l'espressione e calcolata diventa disponibile, a condizione che x non compaia nell'espressione stessa (altrimenti il valore appena calcolato verrebbe subito invalidato dall'assegnamento stesso).

$$gen(x := e) = \{a \in AE \mid x \notin FV(a)\}$$

Equazioni di flusso Per ogni blocco $i \in nodes(G)$ del CFG, si definisce lo stato entrante $entry(i)$ e lo stato uscente $exit(i)$.

Definizione 4.5: Stato entrante

Lo stato entrante di un blocco i è dato dall'intersezione degli stati uscenti di tutti i suoi predecessori:

$$entry(i) = \bigcap \{exit(j) \mid (j, i) \in edges(G)\}$$

la scelta dell'intersezione ($\cap$) è determinata dalla natura **must** dell'analisi: un'espressione è disponibile solo se è disponibile lungo tutti i cammini che raggiungono il blocco.

Nota 4.4: Blocco iniziale

Se i non ha predecessori (blocco iniziale), allora $entry(i) = \emptyset$.

Definizione 4.6: Stato uscente

Lo stato uscente di un blocco i è calcolato applicando la funzione di trasferimento allo stato entrante:

$$exit(i) = (entry(i) \setminus kill(stmt(G, i))) \cup gen(stmt(G, i))$$

Ovvero:

1. Si parte dalle espressioni disponibili in ingresso: $entry(i)$
2. Si rimuovono le espressioni invalidate: $\setminus kill(stmt(G, i))$
3. Si aggiungono le espressioni generate: $\cup gen(stmt(G, i))$

Da queste funzioni è possibile riscrivere le funzioni gen e $kill$ in modo più generale:

- $kill(x := e) = \{a \in AE \mid x \in FV(a)\}$
- $gen(x := e) = \{a \in AE \mid x \notin FV(a)\}$

Definizione 4.7: Funzione di Trasferimento

In un'analisi dataflow, una funzione di trasferimento f_i è una funzione matematica associata a ciascun blocco (o istruzione) i del CFG che descrive come l'esecuzione di quel blocco trasforma lo stato dell'analisi.

Formalmente, dato un reticolo completo L che rappresenta il dominio dell'analisi, la funzione di trasferimento mappa lo stato in ingresso nello stato in uscita:

$$f_i : L \rightarrow L$$

$$exit(i) = f_i(entry(i))$$

Nelle analisi dataflow classiche (es. available expressions), la funzione di trasferimento assume tipicamente la forma:

$$f_i(x) = (x \setminus kill_i) \cup gen_i$$

dove:

- x è lo stato in ingresso ($entry(i)$).
- $kill_i$ è l'insieme degli elementi invalidati dall'istruzione i .
- gen_i è l'insieme degli elementi generati dall'istruzione i .

Nota 4.5: Monotonia

Affinché l'algoritmo iterativo (worklist) converga sicuramente a un punto fisso (terminazione), è richiesto che le funzioni di trasferimento siano monotone rispetto all'ordinamento del reticolo $\sqsubseteq$. Ovvero:

$$x \sqsubseteq y \implies f(x) \sqsubseteq f(y)$$

Le funzioni basate su gen e $kill$ soddisfano sempre questa proprietà.

4.2.2 Altre analisi di dataflow

Come accennato nella nota (Nota 4.2), le analisi di dataflow possono essere classificate secondo vari criteri. In aggiunta all'analisi delle available expressions, esistono numerose altre analisi di dataflow utilizzate per diversi scopi nell'ottimizzazione del codice. Alcuni esempi di altre analisi viste nel corso di linguaggi interpreti e compilatori di dataflow includono:

- **Live variables**
- **Reaching definitions**
- **Very busy expressions**

L'immagine seguente classifica alcune di queste analisi in base alla direzione del flusso e alla semantica della proprietà

	Possible	Definite
Forward	Reaching definitions <i>"For each program point, which assignments may have been made and not overwritten, when program execution reaches this point along some path."</i>	Available expressions <i>"For each program point, which expressions must have already been computed, and not later modified, on all paths to the program point"</i>
Backward	Live variables <i>"For each program point, which variables may be live at the exit from the program point. A variable is live if its content will be read in some path starting from the program point."</i>	Very busy expressions <i>"For each program point, which expressions must be very busy at the exit from the point. An expression is very busy if it will be always used in all paths starting from a label before any of the variables occurring in it are redefined."</i>

Figura 14: L'immagine illustra la classificazione di alcune analisi di dataflow.

Definizione 4.8: Reaching definitions

"For each program point, which assignments may have been made and not overwritten, when program execution reaches this point along some path"

Un'assegnazione $d : x := e$ al blocco i raggiunge (reaches) un punto del programma p se esiste almeno un cammino dal blocco i al punto p lungo il quale la variabile x non viene ridefinita.

Quest'analisi è di tipo **forward** e **may**; l'informazione si propaga in avanti nel CFG ed è sufficiente che un'assegnazione raggiunga il punto lungo almeno un cammino.

Definizione 4.9: Live variables

"For each program point, which variables may be live at the exit from the program point. A variable is live if its content will be read in some path starting from the program point"

Una variabile x è viva (live) in un punto del programma p se esiste almeno un cammino che parte da p lungo il quale il valore di x viene utilizzato prima di essere ridefinito.

Quest'analisi è di tipo **backward** e **may**; l'informazione si propaga all'indietro nel CFG (dai successori ai predecessori) ed è sufficiente che la variabile sia utilizzata lungo almeno un cammino futuro.

Definizione 4.10: Very busy expressions

"For each program point, which expressions must be very busy at the exit from the point. An expression is very busy if it will be always used in all paths starting from a label before any of the variables occurring in it are redefined"

Un'espressione e è molto impegnativa (very busy) in un punto del programma p se lungo ogni cammino che parte da p l'espressione e viene calcolata prima che qualsiasi variabile che occorre in e venga ridefinita.

Quest'analisi è di tipo **backward** e **must**; l'informazione si propaga all'indietro nel CFG e l'espressione deve essere utilizzata lungo tutti i cammini futuri.

4.3 I reticoli nelle analisi di dataflow

Tutte le analisi di dataflow discusse operano su dei reticoli completi (Definizione 2.11).

Definizione 4.11: Reticolo delle analisi dataflow

Ogni analisi dataflow può essere definita come una tupla che rappresenta un reticolo completo (Definizione 2.11). Per le analisi viste, i reticoli sono:

1. **Available expressions:** $\langle \wp(AE), \supseteq, \emptyset, AE, \cap, \cup \rangle$.
2. **Live variables:** $\langle \wp(Var), \subseteq, \emptyset, Var, \cup, \cap \rangle$.
3. **Reaching definitions:** $\langle \wp(Def), \subseteq, \emptyset, Def, \cup, \cap \rangle$.
4. **Very busy expressions:** $\langle \wp(BE), \supseteq, \emptyset, BE, \cup, \cap \rangle$.

Affinché l'algoritmo iterativo termini sicuramente, il reticolo deve soddisfare la ascending chain condition (Definizione 2.19).

Nota 4.6: Modellazione analisi statiche

Non tutte le analisi statiche possono essere modellate come analisi di dataflow.

4.4 Analisi distributive

Le analisi dataflow classiche appartengono alla categoria delle analisi distributive. Questa proprietà matematica è fondamentale perché garantisce che l'analisi possa essere risolta con algoritmi standard efficienti, ma ne limita anche la potenza espressiva.

Definizione 4.12: Distributività

Sia $\langle \wp(X), \subseteq, \emptyset, X, \cup, \cap \rangle$ un reticolo completo (Definizione 2.11) e $f : \wp(X) \rightarrow \wp(X)$ una funzione (Definizione 2.14). f si dice **distributiva** se:

$$f(x \cup y) = f(x) \cup f(y)$$

per ogni $x, y \in \wp(X)$.

Nota 4.7: Intuizione

Quanto descritto nella definizione sopra (Definizione 4.12) in termini intuitivi, significa che il risultato dell'analisi è identico sia che si uniscano le informazioni prima di eseguire l'istruzione, sia che le si uniscano dopo averla eseguita su ogni singolo ramo.

Nota 4.8: La potenza delle analisi distributive

Le analisi distributive tengono traccia di **come** la computazione avviene (es. variabili che sono assegnate ed espressioni che vengono calcolate) all'interno del programma e non del **cosa** il programma calcola.

Per questo motivo le analisi complesse non sono generalmente distributive e molto spesso non possono essere modellate come analisi di dataflow.

4.4.1 Constant propagation

Un esempio di analisi non distributiva è la constant propagation.

Definizione 4.13: Constant propagation

“ For each program point, whether or not a variable has a constant value whenever an execution reaches that point ”

Un reticolo per la constant propagation può essere definito come:

$$CP = \langle \mathbb{Z} \cup \{\perp\} \cup \{\top\}, \sqsubseteq, \perp, \top, \sqcup, \sqcap \rangle$$

dove $\perp$ rappresenta il valore costante di una variabile non definita, $\top$ rappresenta il valore non costante (variabile) e l'operatore di join $\sqcup$ è definito come:

$$c_1 \sqcup c_2 = \begin{cases} c_1 & \text{se } c_1 = c_2 \\ \top & \text{altrimenti} \end{cases}$$

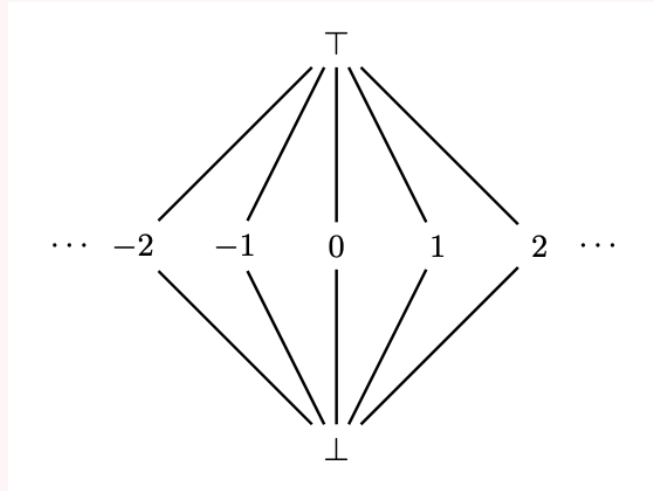


Figura 15: L'immagine illustra il reticolo per la constant propagation.

Nota 4.9: Non distributività della constant propagation

Considerando il seguente frammento di codice:

```
if (condition) then
  y := 1
  z := -1
else
  y := 0
```

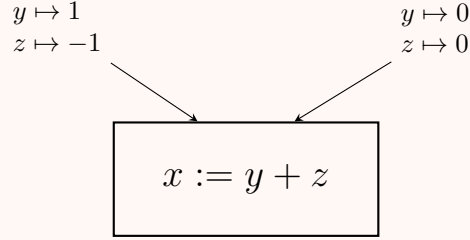
```

    z := 0
end if

x := y + z

```

Da qui è possibile costruire il seguente CFG:



Dal CFG sopra è possibile costruire due insiemi di stati:

- Stato s_0 (nodo sinistro): $\{y \mapsto 1, z \mapsto -1\}$
- Stato s_1 (nodo destro): $\{y \mapsto 0, z \mapsto 0\}$

La funzione di trasferimento al join point (l'istruzione $x := y + z$) calcola il nuovo stato:

$$f(s_{in}) = f(s_0 \sqcup s_1)$$

dove $\sqcup$ è l'operatore di join del reticolo CP.

Per verificare se la constant propagation è distributiva, è necessario calcolare $f(s_0 \sqcup s_1)$ e confrontarlo con $f(s_0) \sqcup f(s_1)$:

Prima di eseguire l'istruzione, l'analisi unisce le informazioni provenienti dai due rami:

- Per y : $1 \sqcup 0 = \top$.
- Per z : $-1 \sqcup 0 = \top$.

Quindi lo stato unito è $s_{join} = [y \mapsto \top, z \mapsto \top]$.

Successivamente l'analisi valuta $x := y + z$ usando lo stato s_{join} :

$$x := \top + \top \implies x \mapsto \top$$

Quindi $f(s_0 \sqcup s_1) = [x \mapsto \top, y \mapsto \top, z \mapsto \top]$, portando al risultato che x non è costante.

Tuttavia, se l'analisi fosse distributiva si dovrebbe ottenere lo stesso risultato unendo i risultati delle singole esecuzioni ($f(s_0) \sqcup f(s_1)$):

- Nel ramo sinistro (s_0): $x := 1 + (-1) = 0$.
- Nel ramo destro (s_1): $x := 0 + 0 = 0$.
- Unione dei risultati: $0 \sqcup 0 = 0$.

Poiché il risultato reale dell'analisi ($\top$) è diverso e meno preciso del risultato ideale (0):

$$f(s_0 \sqcup s_1) \neq f(s_0) \sqcup f(s_1)$$

$$\top \neq 0$$

si dimostra che la propagazione delle costanti non è un'analisi distributiva.

5 Introduzione a LiSA e architettura di un analizzatore statico

5.1 Panoramica di un analizzatore statico

Un analizzatore statico è composto da diversi componenti modulari che interagiscono per trasformare il codice sorgente in un risultato di analisi (es. warning o garanzie di correttezza).

Definizione 5.1: Componenti Principali

Il flusso di un analizzatore statico può essere riassunto nei seguenti passaggi logici:

- **Programma:** Il codice sorgente in input.
- **IR:** Il programma viene tradotto in una rappresentazione interna che è più facile da analizzare.
- **Engine di analisi:** Il cuore dell'analizzatore che gestisce l'algoritmo di punto fisso, la risoluzione delle chiamate e la gestione della memoria.
- **Dominio:** L'astrazione dei dati su cui lavora l'algoritmo (es. propagazione delle costanti, intervalli).

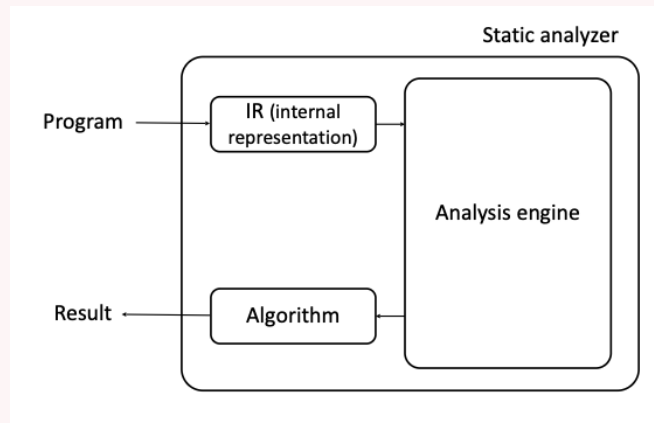


Figura 16: L'immagine illustra i componenti principali di un analizzatore statico.

Nota 5.1: Modularità

Una domanda fondamentale nella progettazione di un analizzatore è: “Come cambiare l’analisi o il linguaggio senza riscrivere tutto?”. La risposta risiede nell’indipendenza dei componenti:

- I componenti dell’analisi devono essere indipendenti l’uno dall’altro.
- I componenti dell’analisi e la IR non devono essere modellati su uno specifico linguaggio sorgente.

Questo permette, ad esempio, di cambiare il dominio astratto (da costanti a intervalli) senza modificare l'algoritmo di visita.

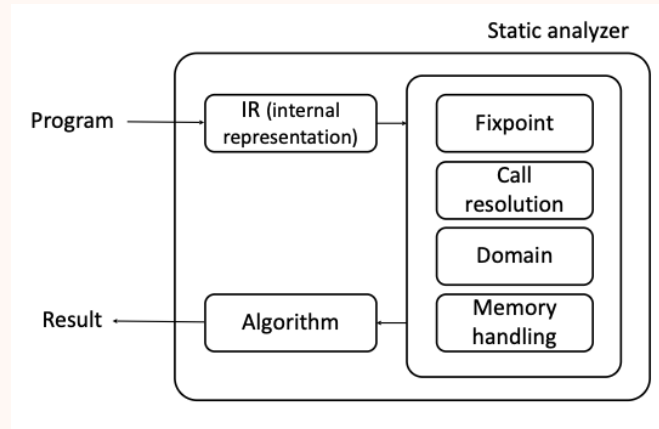


Figura 17: L'immagine illustra una possibile scomposizione modulare di un analizzatore statico.

5.2 LiSA

LiSA (Library for Static Analysis) è una libreria open-source scritta in Java per la realizzazione di analizzatori statici basati su interpretazione astratta.

5.2.1 Overview

In LiSA, il codice sorgente viene tradotto da un front-end specifico per il linguaggio (es. per Java o IMP) in una collezione di control flow graph.

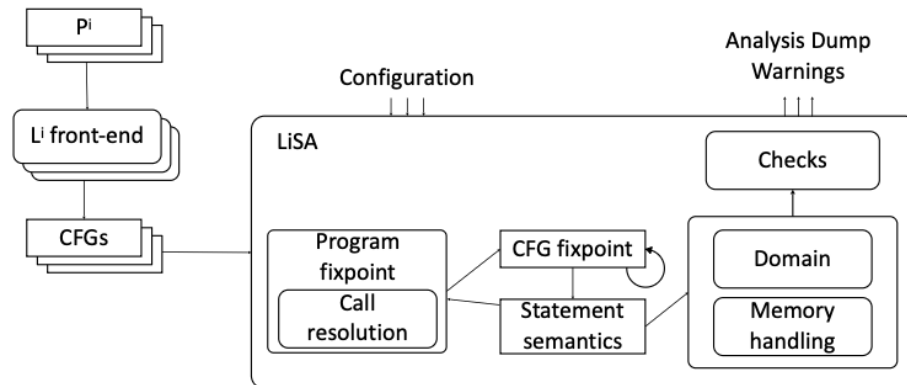


Figura 18: L'immagine illustra il flusso di analisi in LiSA.

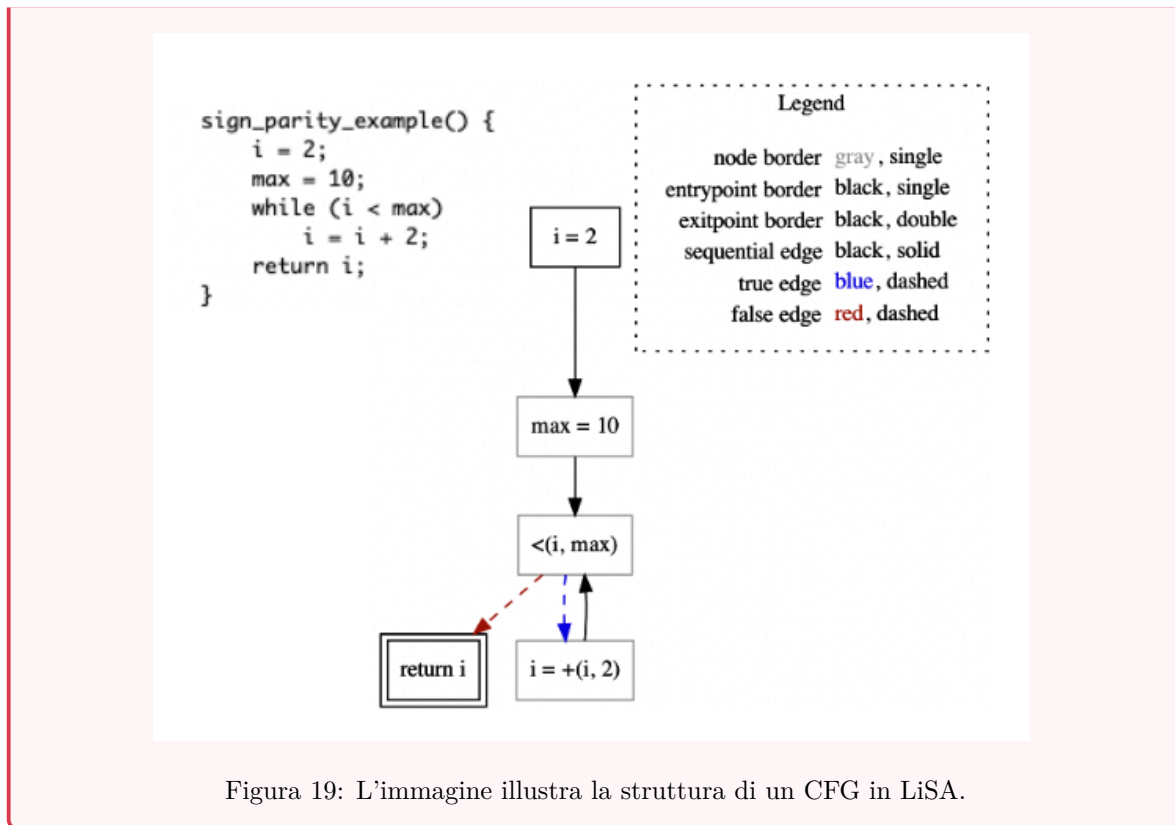
Definizione 5.2: Struttura di un CFG in LiSA

Un CFG in LiSA è costituito da:

- Un insieme di nodi, che rappresentano gli statement del programma.
- Un insieme di archi che collegano i nodi.

Gli archi possono essere di tre tipi:

- **Sequential edge:** Flusso di esecuzione sequenziale standard.
- **True edge:** Preso quando la condizione del nodo sorgente è valutata come vera (assumed to hold).
- **False edge:** Preso quando la condizione del nodo sorgente è valutata come falsa (assumed not to hold).



L'utilizzo dei CFG permette di astrarre dalla sintassi specifica del controllo di flusso del linguaggio originale (es. cicli while, for, if) e definire algoritmi di punto fisso direttamente sul grafo.

5.3 Sintassi vs Semantica

Un concetto chiave in LiSA è la distinzione tra lo statement sintattico (il nodo del CFG) e il suo significato semantico.

Nota 5.2: Il problema della sintassi

Uno statement potrebbe essere complesso e avere diverse rappresentazioni sintattiche equivalenti oppure diverse operazioni sintattiche possono avere la stessa semantica. Ad esempio, in Java, "a" + "b" e "a".concat ("b") eseguono entrambi una concatenazione di stringhe. Un dominio astratto non dovrebbe preoccuparsi di come l'operazione è scritta sintatticamente.

Per risolvere questo problema, LiSA utilizza un linguaggio interno di **symbolic expressions**. Così facendo:

- I domini lavorano su espressioni simboliche, non sugli statement grezzi.
- Ogni espressione simbolica rappresenta un'operazione semantica "atomica" (es. somma aritmetica, concatenazione stringhe).

- Durante il calcolo del punto fisso, il metodo `Statement.semantics()` riscrive lo statement in una o più espressioni simboliche da passare al dominio.

Esempio 5.1: Rewriting semantico

Consideriamo un operatore `Plus` generico. La sua semantica potrebbe essere definita come:

- Se gli operandi sono stringhe $\rightarrow$ ritorna una `StringConcat`.
- Altrimenti $\rightarrow$ ritorna una `ArithmSum`.

Il dominio astratto implementerà poi la logica specifica per `StringConcat` o `ArithmSum`.

```
Plus.semantics():  
    if left is string || right is string:  
        return domain.eval(new StringConcat(left, right))  
    else:  
        return domain.eval(new ArithmSum(left, right))
```

Figura 20: L'immagine illustra lo pseudocodice dell'operatore `Plus`.

5.4 Implementazione dell'analisi dataflow

LiSA fornisce un'architettura specifica per implementare le analisi dataflow (Vedi 4.2).

Definizione 5.3: Dataflow in LiSA

Le analisi dataflow in LiSA sfruttano l'algoritmo di punto fisso generico basato su worklist.

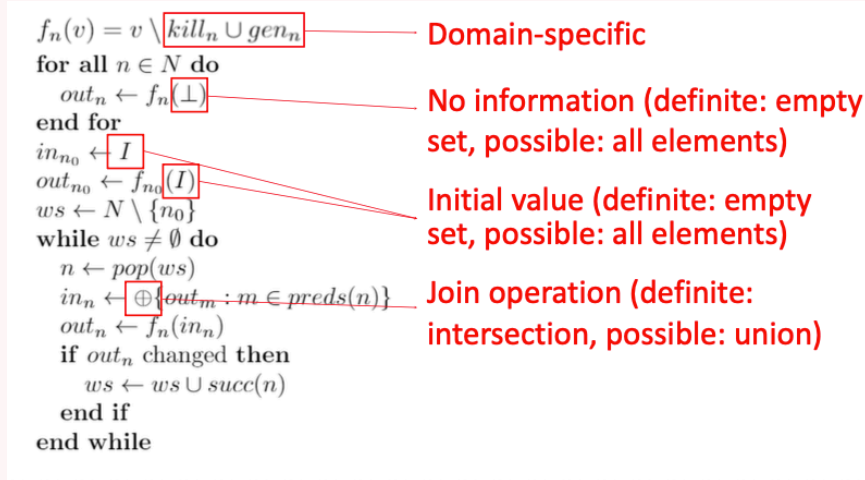
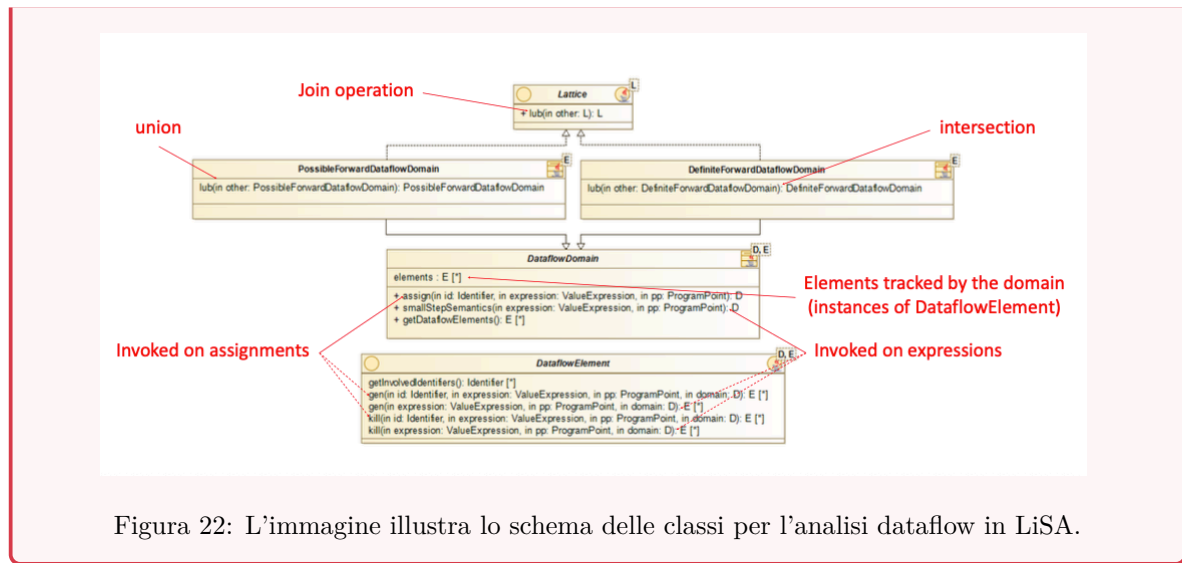


Figura 21: L'immagine illustra l'algoritmo di punto fisso generico basato su worklist per l'analisi dataflow forward, evidenziando in rosso i componenti che variano in base alla semantica dell'analisi (May vs Must) e che necessitano di essere modularizzati.

L'implementazione è divisa in due classi principali per separare la gestione del reticolo dalla logica di trasferimento:

- **DataflowDomain:** Gestisce la logica generale del dominio, inclusa la valutazione delle espressioni e degli assegnamenti. Esistono due specializzazioni principali basate sul tipo di join:
 - **PossibleForwardDataflowDomain** che implementa un'analisi *May* (possibile). L'operazione di lub (join) è l'unione degli insiemi.
 - **DefiniteForwardDataflowDomain** che implementa un'analisi *Must* (definita). L'operazione di lub (join) è l'intersezione degli insiemi.
- **DataflowElement:** Rappresenta il singolo elemento tracciato dal dominio. Questa classe definisce le operazioni specifiche di **gen** e **kill**.



Nota 5.3: Tipi di analisi supportate

Attualmente, LiSA supporta esclusivamente analisi di tipo **forward**. Per quanto riguarda la semantica dell'analisi, sono supportate sia le analisi *May* (possibile) che *Must* (definita).

6 Il significato dell'approssimazione

6.1 Intuizione e necessità dell'approssimazione

Considerando una proprietà semantica concreta di un programma, denotata come $[[P]]$, e una proprietà target Q che si vuole verificare. L'obiettivo ideale sarebbe verificare se $[[P]] \subseteq Q$. Tuttavia, a causa del Teorema di Rice e dell'indecidibilità dell'arresto, questa inclusione è in generale indecidibile. Per questo motivo si ricorre ad un'approssimazione $[[P]]^\#$ di $[[P]]$.

Definizione 6.1: Approssimazione

Per rendere il problema decidibile, si calcola un'approssimazione $[[P]]^\#$ della semantica concreta $[[P]]$ tale che rispetti due proprietà:

- **Soundness (Correttezza):** $[[P]] \subseteq [[P]]^\#$. L'approssimazione deve includere tutti i comportamenti reali del programma.
- **Decidability (Decidibilità):** La verifica $[[P]]^\# \subseteq Q$ deve essere decidibile.

Nota 6.1: L'importanza della soundness

La condizione di soundness è cruciale (Definizione 6.1). Se si riesce a dimostrare che l'approssimazione è sicura ($[[P]]^\# \subseteq Q$), allora per la proprietà di transitività dell'inclusione, è concludere che anche il programma reale è sicuro ($[[P]] \subseteq Q$).

Tuttavia, se l'approssimazione non soddisfa la proprietà ($[[P]]^\# \not\subseteq Q$), non è possibile concludere nulla con certezza; potrebbe trattarsi di un vero bug o di un falso allarme dovuto all'imprecisione dell'astrazione ("I don't know").

6.2 Astrazione

Definizione 6.2: Astrazione

L'astrazione è il processo di sostituzione di un oggetto concreto con una descrizione che si focalizza solo su alcune proprietà di interesse, ignorandone altre.

Un'astrazione in un dominio di proprietà $\wp(\Sigma)$ di oggetti in Σ è un sottoinsieme $A \subseteq \wp(\Sigma)$ tale che:

- Gli elementi in A sono descritti con precisione ("senza perdita di precisione").
- Gli elementi non in A devono essere rappresentati (approssimati) da elementi di A , introducendo una perdita di precisione.

6.2.1 Oggetti

Quando si analizza un programma è necessario considerare oggetti che rappresentano parti dello stato di esecuzione. Tra questi è possibile includere:

- Valori delle variabili (interi, booleani, puntatori, ecc.).
- Variabili stesse (nomi, indirizzi, ecc.).
- Stack di esecuzione (chiamate di funzione, contesti, ecc.).
- Environment (mappature tra variabili e valori).
- ...

6.2.2 Proprietà e reticolo delle proprietà

Definizione 6.3: Proprietà

Le proprietà sono insiemi (Definizione 2.1) di oggetti (valori, variabili, stack, ecc.) che condividono una certa caratteristica.

Esempio 6.1: Esempio di proprietà

Consideriamo il dominio degli interi $\mathbb{Z}$. Una proprietà potrebbe essere l'insieme dei numeri pari:

$$P = \{x \in \mathbb{Z} \mid x \bmod 2 = 0\}$$

Definizione 6.4: Reticolo delle proprietà

L'insieme di tutte le possibili proprietà $\wp(\Sigma)$ di oggetti in Σ è un reticolo distributivo completo definito come:

$$\langle \wp(\Sigma), \subseteq, \emptyset, \Sigma, \cup, \cap, \neg \rangle$$

dove:

- Una proprietà è un sottoinsieme di Σ
- L'ordine parziale $\subseteq$ è dato dalle implicazioni logiche tra le proprietà.
- L'elemento minimo è $\emptyset$ e rappresenta la proprietà falsa (nessun oggetto soddisfa la proprietà).
- L'elemento massimo è Σ e rappresenta la proprietà vera (tutti gli oggetti soddisfano la proprietà).
- L'operazione di join è l'unione di insiemi ($\cup$).
- L'operazione di meet è l'intersezione di insiemi ($\cap$).
- La negazione $\neg$ è il complemento rispetto a Σ ($\neg P = \Sigma \setminus P$).

6.3 Direzione dell'astrazione

Quando si approssima una proprietà concreta P con una astratta $P^\#$, è possibile procedere in due direzioni:

1. **Approssimazione dal basso (from below):** $P^\# \subseteq P$.
2. **Approssimazione dall'alto (from above):** $P \subseteq P^\#$.

6.3.1 Approssimazione dal basso

Nel caso di approssimazione dal basso, $P^\# \subseteq P$ se un elemento appartiene a $P^\#$, è certo che appartenga a P (“Yes”), ma se non appartiene, non sappiamo (“I don’t know”).

Questo approccio è utile per dimostrare la presenza di bug (test), ma non la loro assenza.

Esempio 6.2: Approssimazione dal basso

Nell'approssimazione dal basso, l'insieme astratto $P^\#$ è contenuto interamente nell'insieme concreto P . Formalmente, vale la relazione:

$$P^\# \subseteq P$$

L'obiettivo è rispondere alla domanda se una coppia $\langle x, y \rangle$ appartiene all'insieme concreto P utilizzando solo l'informazione fornita dall'approssimazione $P^\#$. In questa configurazione, si presentano due casi distinti:

- **Caso 1:** $\langle x, y \rangle \notin P^\#$. Se l'elemento non si trova nell'insieme astratto, non si può concludere nulla rispetto all'insieme concreto. Potrebbe trovarsi in P (ma fuori da $P^\#$) oppure essere completamente esterno a P . La risposta è quindi: **“I don’t know”**.
- **Caso 2:** $\langle x, y \rangle \in P^\#$. Se l'elemento si trova nell'insieme astratto, poiché $P^\#$ è un sottoinsieme di P , si è certi che l'elemento appartenga anche all'insieme concreto. La risposta è quindi: **“Yes”**.

Questa tecnica è utile per dimostrare la presenza di una proprietà (o di un bug), ma non la sua assenza (completezza ma non correttezza nel senso della verifica formale classica).

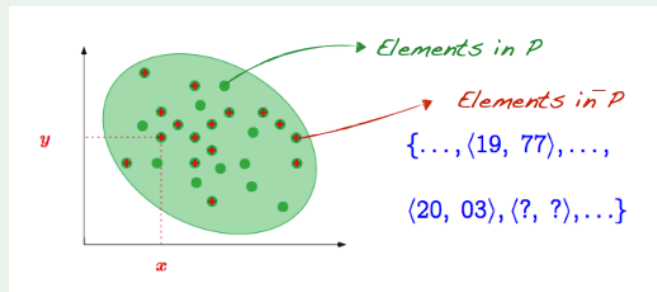


Figura 23: L'immagine illustra il concetto di approssimazione dal basso, dove l'insieme astratto $P^\#$ è un sottoinsieme dell'insieme concreto P .

6.3.2 Approssimazione dall'alto

L'approccio di approssimazione dall'alto, $P \subseteq P^\#$, è la direzione standard per la verifica di sicurezza (safety). Se un elemento non è in $P^\#$, sicuramente non è in P ("No"). Se è in $P^\#$, potrebbe essere in P oppure essere un falso positivo.

Esempio 6.3: Approssimazione dall'alto

Nell'approssimazione dall'alto, l'insieme astratto $P^\#$ è un sovrainsieme dell'insieme concreto P . Formalmente, vale la relazione:

$$P \subseteq P^\#$$

Questa è la forma di approssimazione standard per la verifica di sicurezza (safety). L'obiettivo è rispondere alla domanda se una coppia $\langle x, y \rangle$ appartiene all'insieme concreto P utilizzando solo l'informazione fornita dall'approssimazione $P^\#$. In questa configurazione, si presentano due casi distinti:

- **Caso 1:** $\langle x, y \rangle \notin P^\#$. Se l'elemento non appartiene all'insieme astratto (che è più grande), non può appartenere nemmeno all'insieme concreto P . La risposta è quindi: **"No"**. Questo è il caso più potente: permette di escludere comportamenti indesiderati con certezza.
- **Caso 2:** $\langle x, y \rangle \in P^\#$. Se l'elemento appartiene all'insieme astratto, potrebbe appartenere a P oppure essere un elemento "spurio" aggiunto dall'astrazione (un falso positivo). La risposta è quindi: **"I don't know"**.

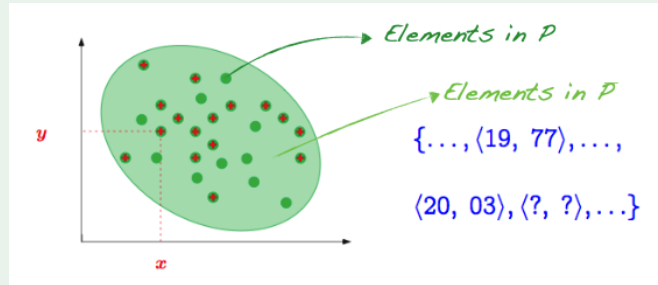


Figura 24: L'immagine illustra il concetto di approssimazione dall'alto, dove l'insieme astratto $P^\#$ è un sovrainsieme dell'insieme concreto P .

Nota 6.2: Correttezza dell'approssimazione dall'alto

Questo approccio garantisce la correttezza (soundness): se l'analisi dice che un errore non può verificarsi (non è in $P^\#$), allora l'errore non si verifica davvero nel programma concreto.

6.4 Esempi di domini Astratti

L'efficacia dell'analisi dipende dalla scelta del dominio astratto. Ecco alcuni esempi classici applicati alle variabili intere:

6.4.1 Dominio dei segni

Il programma si occupa della manipolazione dei numeri interi e si vuole studiare le proprietà relative ai segni (positivo, negativo, zero) dei valori delle variabili. La semantica concreta si occupa delle operazioni standard che manipolano i valori interi (somma, sottrazione, ecc.), mentre la semantica astratta si concentra sulla manipolazione dei segni attraverso regole astratte.

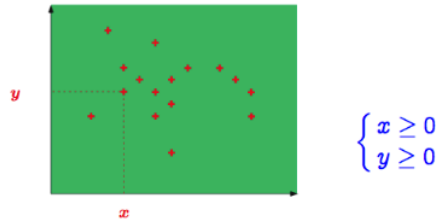


Figura 25: L'immagine illustra il dominio astratto dei segni.

Il dominio concreto studiato è $\wp(\mathbb{Z})$ e la proprietà che si vuole studiare è il segno delle variabili. Il dominio astratto (Definizione 6.2) dei segni $D^\#$ è costituito dai seguenti elementi:

- $(+)$ che rappresenta i numeri positivi (> 0), ovvero l'insieme $\{1, 2, 3, \dots\}$
- $(-)$ che rappresenta i numeri negativi (< 0), ovvero l'insieme $\{-1, -2, -3, \dots\}$
- (0) che rappresenta il numero zero, ovvero l'insieme $\{0\}$
- $\perp$ rappresenta l'insieme vuoto $\emptyset$ (nessun valore possibile, codice irraggiungibile).
- $\top$ rappresenta l'intero insieme $\mathbb{Z}$ (qualsiasi valore intero è possibile, “non so”).

Relazione di astrazione Un insieme X in $\wp(\mathbb{Z})$ è approssimato dal più piccolo insieme di interi che contiene X ed è rappresentabile nel dominio dei segni. Formalmente, questo forma un reticolo completo (Definizione 2.11) dove l'ordine parziale $\sqsubseteq$ riflette l'inclusione insiemistica delle concretizzazioni.

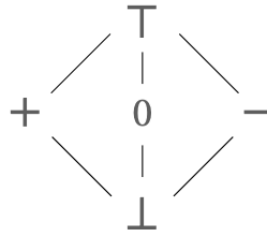


Figura 26: L'immagine illustra il reticolo del dominio dei segni.

Il reticolo rappresentato in Figura 26 mostra che gli elementi $+$, 0 e $-$ sono incomparabili tra loro, ma sono tutti “sotto” $\top$ e “sopra” $\perp$.

Nota 6.3: Limitazioni e operazioni astratte

Il dominio dei segni è molto astratto e perde molte informazioni rispetto al dominio concreto o ad altri domini come gli intervalli. Le operazioni devono essere ridefinite per lavorare su questi simboli. Ad esempio:

- $(+) + (+) = (+)$ (La somma di due positivi è positiva)
- $(+) + (-) = \top$ (La somma di un positivo e un negativo è sconosciuta nel dominio dei segni, poiché il risultato dipende dai valori assoluti che sono stati astratti).

Questo dimostra come l'astrazione garantisca la terminazione e la decidibilità, pagando il prezzo della precisione (restituendo $\top$ quando non può determinare un segno preciso).

6.5 Intervalli

6.6 Ottogoni

6.7 Poliedri

6.8 Approssimazione delle computazioni

Oltre ad astrarre i dati (valori delle variabili), l'analisi statica astrae il concetto stesso di esecuzione del programma.

L'obiettivo è passare da una visione basata su singole esecuzioni (vedi 3.4) a una visione basata su insiemi di stati calcolabili.

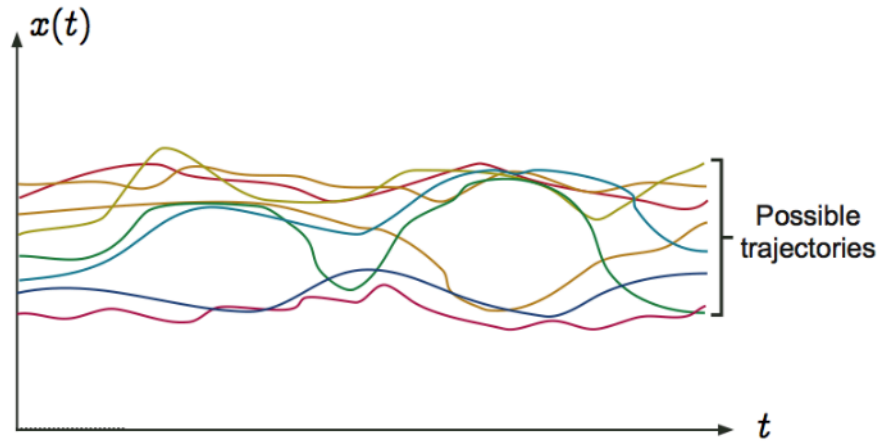


Figura 27: L'immagine illustra il processo di astrazione delle esecuzioni del programma.

Questo processo di astrazione avviene attraverso diversi stadi concettuali fondamentali:

- **Small-step transition semantics**
- **Computing on sets** (Calcolo su insiemi)
- **Computing by Fixpoint** (Calcolo tramite punto fisso)
- **Collecting semantics** (Semantica collettiva)

6.8.1 Small-step transition semantics

In questa fase, invece di considerare l'intera storia temporale continua $x(t)$ di un'esecuzione, viene considerato il comportamento del programma in singoli passi di transizione. Questo approccio definisce le regole operazionali (semantica operativa) che permettono di passare da uno stato corrente al successivo, creando una traiettoria discreta di stati.

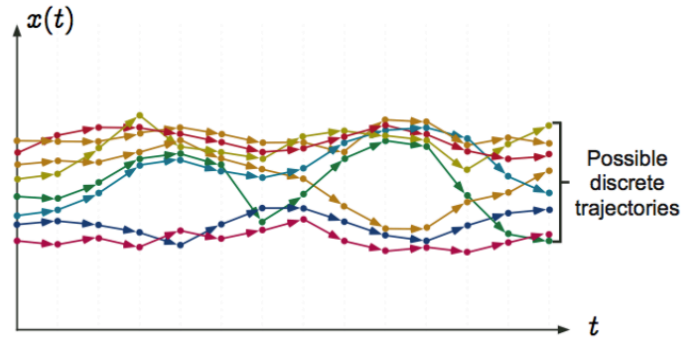


Figura 28: L'immagine illustra la transizione tra stati discreti nel tempo durante l'esecuzione di un programma.

6.8.2 Computing on sets (Calcolo su insiemi)

Invece di analizzare una singola traiettoria (visualizzabile come un singolo puntino che si muove nel grafico degli stati nel tempo), l'analisi considera l'evoluzione di un insieme di stati simultaneamente. Ad ogni istante discreto, viene considerato l'insieme di tutti i possibili stati in cui il programma potrebbe trovarsi dato un insieme di stati iniziali.

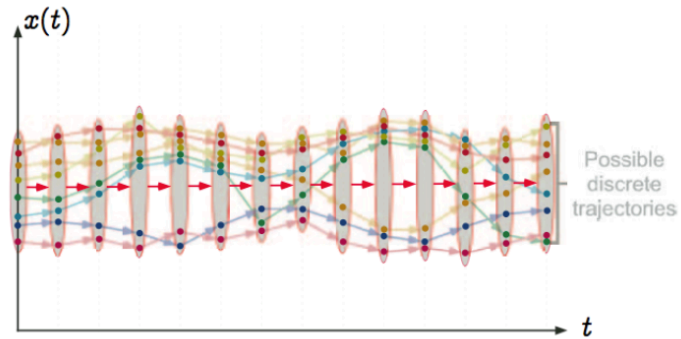


Figura 29: L'immagine illustra l'evoluzione di un insieme di stati nel tempo durante l'esecuzione di un programma.

6.8.3 Computing by fixpoint (Calcolo tramite punto fisso)

Per calcolare l'insieme di tutti gli stati raggiungibili (che potrebbero evolvere all'infinito nel tempo), viene utilizzato un approccio iterativo. Partendo dagli stati iniziali (spesso $\perp$ o l'insieme degli stati di ingresso) (vedi 30) viene applicata ripetutamente la funzione di transizione all'interno dell'insieme.

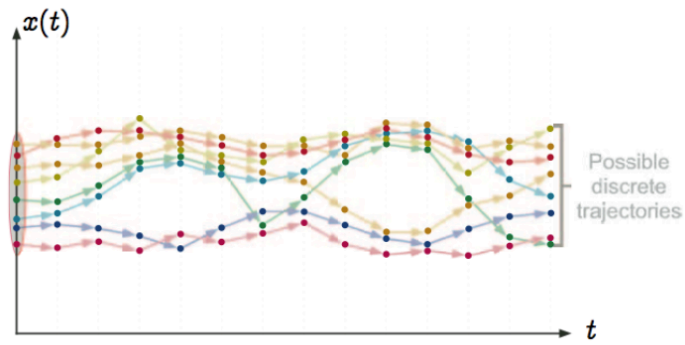


Figura 30: L'immagine illustra lo stato iniziale dell'iterazione per il calcolo del punto fisso.

L'insieme degli stati accumulati cresce (vedi 31) fino a quando non si stabilizza, ovvero quando un'ulteriore applicazione della funzione non aggiunge nuovi stati.

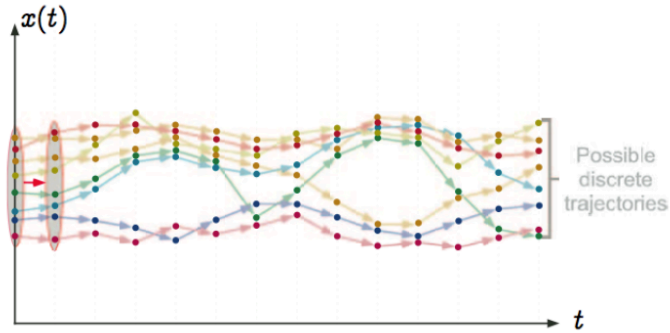


Figura 31: L'immagine illustra l'espansione progressiva dell'insieme degli stati raggiungibili attraverso l'applicazione iterativa della funzione di transizione.

Questo stato stabile (vedi 32) è il punto fisso (vedi 2.16).

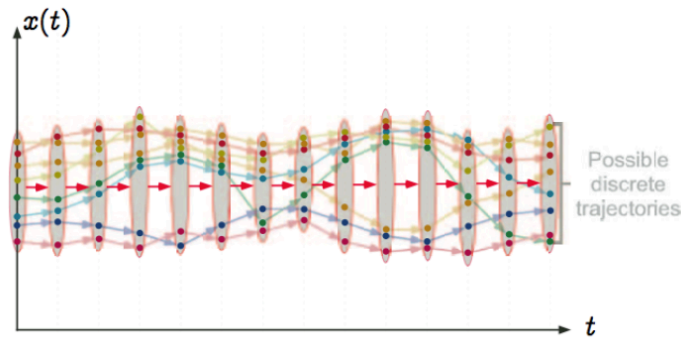


Figura 32: L'immagine illustra il raggiungimento del punto fisso: l'insieme degli stati si è stabilizzato e include tutte le possibili traiettorie discrete.

6.8.4 Collecting semantics (Semantica collettrice)

La semantica collettrice associa ad ogni punto del programma l'insieme di tutti gli stati che possono essere raggiunti in quel punto durante qualsiasi esecuzione senza tenere conto dell'ordine temporale (questa è già una astrazione).

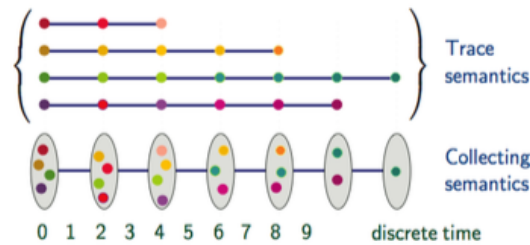


Figura 33: L'immagine illustra la semantica collettrice che associa insiemi di stati ai punti del programma.

Nota 6.4: Introduzione di rumore

Il passaggio alla semantica collettrice introduce “rumore” o approssimazione. Associando gli stati ai punti di programma (es. riga di codice) e non al tempo, potrebbe includere percorsi spuri (spurious traces) che saltano tra stati validi in modi che l'esecuzione reale non farebbe, sebbene rimanga limitato agli stati raggiungibili.

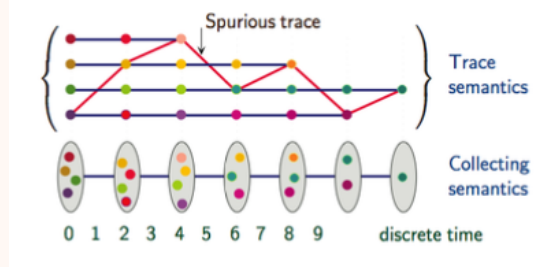


Figura 34: L'immagine illustra l'introduzione di rumore nella semantica collettrice a causa della perdita di informazioni temporali.

L'analisi statica procede quindi:

- Considerando proprietà (insiemi) di stati invece di stati concreti.
- Calcolando su insiemi (approssimando la semantica delle tracce).
- Utilizzando la collecting semantics sui punti di programma.

Così facendo però rimane ancora il problema della undecidibilità, poiché gli insiemi di stati possono essere infiniti e non calcolabili. Infatti è necessario approssimare ulteriormente questi insiemi.

6.8.5 Computing on properties (Calcolo su proprietà)

Il calcolo esatto sugli insiemi di stati nella semantica collettrice è spesso indecidibile o computazionalmente intrattabile (gli insiemi possono essere infiniti o estremamente complessi).

Per risolvere questo problema, viene introdotta un'ulteriore astrazione; invece di propagare l'esatto insieme di stati concreti, si calcolano delle proprietà astratte che sovrapprossimano tali insiemi.

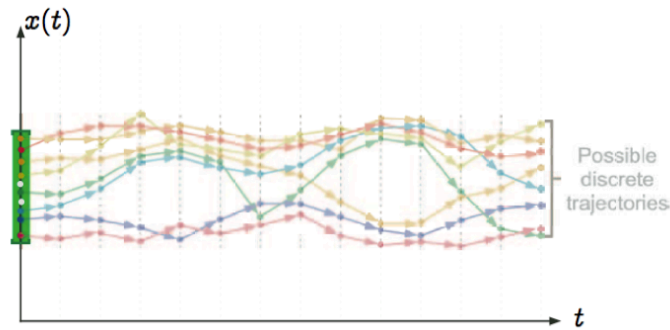


Figura 35: L'immagine illustra l'astrazione dei dati: un insieme di stati concreti (i punti colorati all'istante iniziale) viene sostituito da una singola proprietà astratta (l'intervallo verde) che li include tutti.

In questa fase:

- Si sostituiscono gli insiemi di stati concreti con elementi di un dominio astratto (es. Segni, Intervalli, Poliedri).
- Le operazioni del programma vengono sostituite da operazioni astratte che lavorano su queste proprietà.

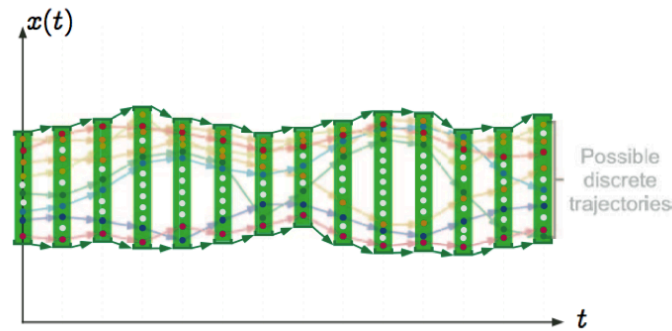


Figura 36: L'immagine illustra l'esecuzione astratta: la computazione non avviene più sui singoli stati, ma evolve trasformando la proprietà astratta (la sequenza di intervalli verdi) passo dopo passo, garantendo di coprire tutte le possibili traiettorie discrete.

Nota 6.5: Aggiunta di rumore

Questo passaggio introduce ulteriore rumore (perdita di precisione). Ad esempio, un insieme di stati concreti sparsi potrebbe essere sovrapprossimato da un singolo intervallo astratto che include anche stati non spuri.

6.9 Gerarchia delle semantiche come astrazioni

È possibile visualizzare l'intero processo di costruzione dell'analisi statica come una gerarchia di astrazioni successive, dove ogni livello riduce la precisione per guadagnare trattabilità o focalizzarsi su proprietà specifiche.

Le figure seguenti mostrano come, partendo dalla stessa realtà concreta, si possano derivare astrazioni diverse.

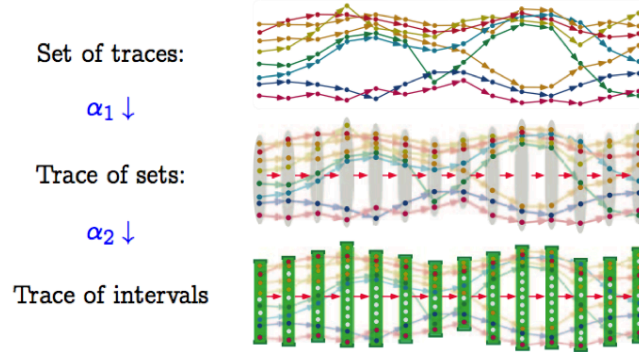


Figura 37: Astrazione dei dati (α_2): dalle tracce concrete agli intervalli, mantenendo la distinzione temporale.

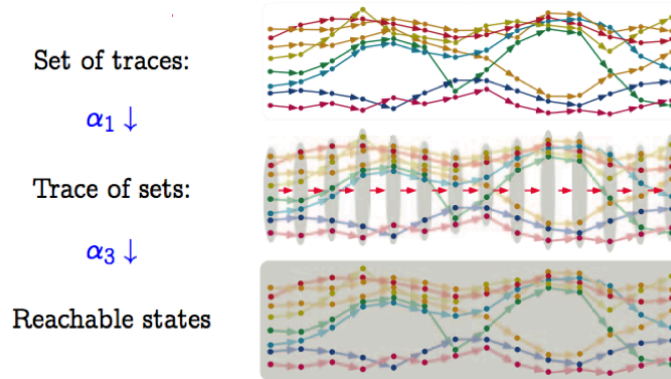


Figura 38: Astrazione degli stati raggiungibili (α_3): dalle tracce concrete all'insieme globale degli stati visitati.

La gerarchia si compone dei seguenti livelli:

1. **Set of traces (Insieme di tracce)**: Rappresenta la realtà completa del programma (semantica concreta). Distingue ogni singola esecuzione e ogni singolo istante temporale. È la massima precisione possibile, ma il calcolo è generalmente impossibile.
2. **Trace of sets (Traccia di insiemi)** [α_1]: Ottenuta tramite la prima astrazione (semantica colletttrice). Qui si raggruppano le esecuzioni diverse: invece di seguire singole linee, si considerano "fasci" (insiemi) di stati per ogni istante discreto. Si perde la relazione tra uno stato specifico al tempo t e il suo successore al tempo $t + 1$, mantenendo solo l'insieme dei possibili stati in t e in $t + 1$.
3. **Ulteriori astrazioni** [α_2 vs α_3]: A partire dalla traccia di insiemi, possiamo astrarre in modi diversi a seconda dello scopo:

- **Trace of intervals** (α_2 - **Figura 37**): Si applica un'astrazione sui **dati**. Invece di insiemi di punti sparsi, si usano domini astratti geometrici (come gli intervalli verdi) per inglobare tutti i valori possibili a ogni passo. Mantiene la struttura temporale discreta.
- **Reachable states** (α_3 - **Figura 38**): Si astrae la dimensione **temporale**. Invece di distinguere cosa accade al passo 1, 2 o 3, si calcola l'unico grande insieme (l'area grigia) che contiene tutti gli stati che il programma può raggiungere in qualsiasi momento della sua esecuzione.

7 Galois connections

7.1 Dominio concreto

Il dominio concreto C rappresenta l'universo dei valori reali che il programma può assumere. In questo caso viene considerato il dominio concreto come l'insieme delle parti dell'insieme dei numeri interi.

Definizione 7.1: Lattice del Dominio Concreto

Il dominio concreto $\langle \wp(\mathbb{Z}), \subseteq, \cap, \cup, \mathbb{Z}, \emptyset \rangle$ forma un reticolo completo (Definizione 2.11), dove:

- L'ordine parziale è l'inclusione insiemistica ($\subseteq$).
- $S_1 \subset S_2$ indica che S_1 è “più preciso” di S_2 .
- $S_1 \not\subseteq S_2 \wedge S_2 \not\subseteq S_1$ indica che S_1 e S_2 non sono comparabili.
- Il join ($\cup$) rappresenta l'unione (o “incertezza”), x deve essere in S_1 oppure in S_2 .
- Il meet ($\cap$) rappresenta l'intersezione, x deve essere in S_1 e in S_2 .
- Il Top ($\top = \mathbb{Z}$) rappresenta “qualsiasi valore” (massima incertezza, es. gli input esterni).
- Il Bottom ($\perp = \emptyset$) rappresenta “nessun valore” (codice irraggiungibile).

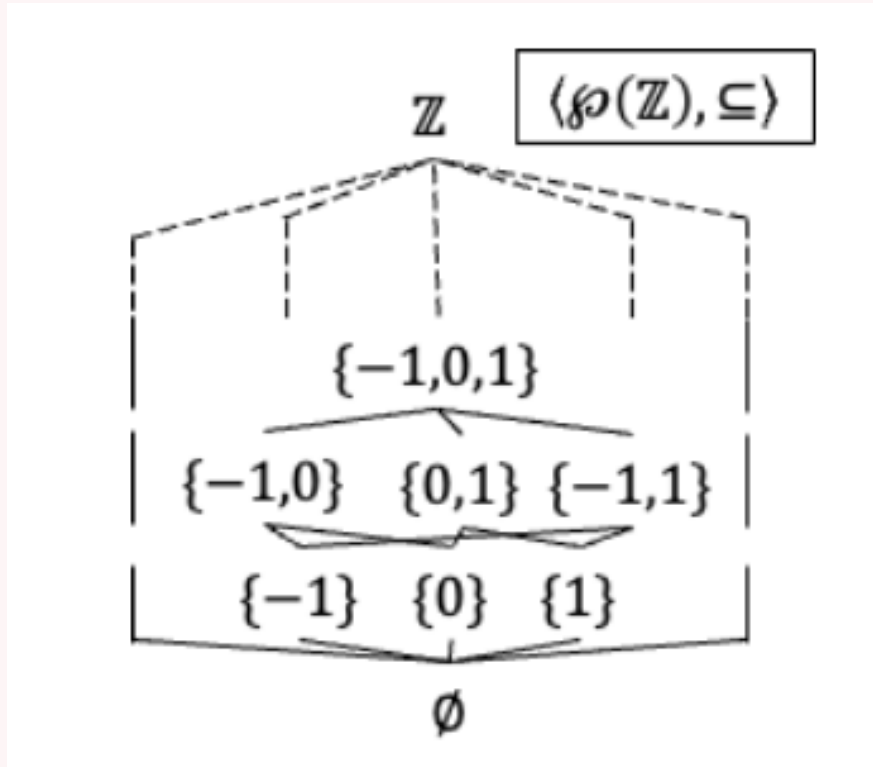


Figura 39: L'immagine illustra il reticolo completo del dominio concreto.

7.1.1 Dominio Astratto

Il dominio astratto A traccia un tipo specifico di informazione (es. segni, intervalli, parità). Esiste una relazione formale tra un dominio concreto C e un dominio astratto A mediata da due funzioni:

- Funzione di astrazione $\alpha : C \rightarrow A$: mappa un insieme di valori concreti nel corrispondente valore astratto.
- Funzione di concretizzazione $\gamma : A \rightarrow C$: mappa un valore astratto nell'insieme di valori concreti che esso rappresenta.

Nota 7.1: Gerarchie tra domini

I domini astratti possono essere organizzati in una gerarchia e possono essere combinati per formare domini più complessi. Questi argomenti verranno trattati in seguito.

7.2 Intuizione sulla soundness dei domini astratti

Affinché un dominio astratto sia corretto (sound), il passaggio tra concreto e astratto deve preservare la correttezza dell'informazione.

Partendo da un valore concreto $c \in C$:

1. L'astrazione: $a = \alpha(c)$.
2. La concretizzazione: $c' = \gamma(a) = \gamma(\alpha(c))$.

Per avere soundness, il risultato c' deve essere una sovra-approssimazione (o uguale) del valore di partenza c . Ovvero, nel processo di astrazione si può perdere precisione (ottenere un insieme più grande), ma non si possono perdere valori originali.

Analogamente, partendo da un valore astratto $a \in A$:

1. La concretizzazione: $c = \gamma(a)$.
2. La ri-astrazione: $a' = \alpha(c) = \alpha(\gamma(a))$.

Il risultato a' deve essere più preciso (o uguale) rispetto ad a ($a' \sqsubseteq_A a$). Concretizzare non perde informazione, quindi ri-astrarre dovrebbe restituire qualcosa di idealmente identico o migliore.

7.3 Monotonia

Le funzioni α e γ devono essere monotone (devono preservare l'ordine).

Teorema 7.1: Monotonia di α

α è monotona: $c_1 \subseteq c_2 \Rightarrow \alpha(c_1) \sqsubseteq_A \alpha(c_2)$.

Se si astrae qualcosa che rappresenta meno informazione (insieme più piccolo), si ottiene un elemento astratto con meno o uguale informazione.

Teorema 7.2: Monotonia di γ

γ è monotona: $a_1 \sqsubseteq_A a_2 \Rightarrow \gamma(a_1) \subseteq \gamma(a_2)$.

Se si concretizza qualcosa di più preciso, si ottiene un insieme concreto più piccolo.

7.4 Galois Connection

La Galois Connection (connessione di Galois) è la formalizzazione algebrica che lega dominio concreto e astratto garantendo le proprietà di soundness e best abstraction.

Definizione 7.2: Galois Connection

Siano $\langle C, \sqsubseteq_C \rangle$ e $\langle A, \sqsubseteq_A \rangle$ due posets. Due funzioni monotone $\alpha : C \rightarrow A$ e $\gamma : A \rightarrow C$ formano una Galois Connection se soddisfano le seguenti proprietà:

- $\gamma \circ \alpha$ è estensiva: $\forall c \in C. c \subseteq_C \gamma(\alpha(c))$.
- $\alpha \circ \gamma$ è riduttiva: $\forall a \in A. \alpha(\gamma(a)) \sqsubseteq_A a$.

Nota 7.2: Interpretazione delle proprietà di GC

Le proprietà di estensività e riduttività garantiscono che:

- Quando si astrae e poi concretizza ($\gamma \circ \alpha$), si ottiene una sovra-approssimazione del valore originale (perdita di precisione).
- Quando si concretizza e poi si astrae, si ottiene un valore astratto più preciso o uguale (nessuna perdita di informazione).

7.5 Galois Insertion (GI)

Una Galois Connection (Definizione 7.2) può contenere elementi nell'astratto che sono ridondanti (più elementi astratti rappresentano lo stesso oggetto concreto) Quando viene rimossa questa ridondanza, si ottiene una Galois Insertion (Intersezione di Galois).

Definizione 7.3: Galois Insertion

Siano $\langle C, \sqsubseteq_C \rangle$ e $\langle A, \sqsubseteq_A \rangle$ due posets e $\alpha : C \rightarrow A$ e $\gamma : A \rightarrow C$ due funzioni monotone. Supponendo che esse formino una Galois Connection (Definizione 7.2) tra C e A , se $\alpha \circ \gamma$ è l'identità su A questa si chiama Galois Insertion.

Nota 7.3: Riduzione da GC a GI

Ogni GC può essere ridotta a una GI rimuovendo gli elementi ridondanti dal dominio astratto (domain reduction). Se non si ha una GI, significa che esistono almeno due elementi astratti che rappresentano la stessa informazione concreta.

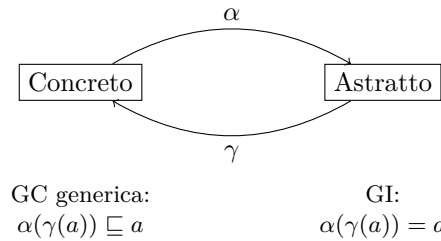


Figura 40: Relazione tra domini in una GC e in una GI.

7.6 Best abstraction

Siano $\langle C, \sqsubseteq_C \rangle$ e $\langle A, \sqsubseteq_A \rangle$ due posets, e $\alpha : C \rightarrow A$ e $\gamma : A \rightarrow C$ due funzioni monotone. Queste formano una Connessione di Galois, se:

$$\forall c \in C, \forall a \in A. \alpha(c) \sqsubseteq_A a \iff c \sqsubseteq_C \gamma(a)$$

dove:

- $\alpha(c) \sqsubseteq_A a \implies c \sqsubseteq_C \gamma(a)$ assicura la soundness
- $c \sqsubseteq_C \gamma(a) \implies \alpha(c) \sqsubseteq_A a$ assicura la best abstraction.

7.6.1 Problema della best abstraction

Non sempre esiste una Galois Connection tra un dominio concreto e un dominio astratto arbitrario.

Esempio 7.1: Poliedri convessi e cerchi

Considerando l'insieme $P = \{(x, y) | x^2 + y^2 \leq 1\}$ (un cerchio). Se il dominio astratto è quello dei poliedri convessi ($a \cdot x + b \cdot y \leq c$), non esiste la best abstraction per il cerchio. È approssimare il cerchio con un quadrato, un ottagono, un dodecagono, etc., ma è sempre possibile trovare un poliedro con più lati che approssima il cerchio meglio del precedente senza mai coincidere con il cerchio stesso.

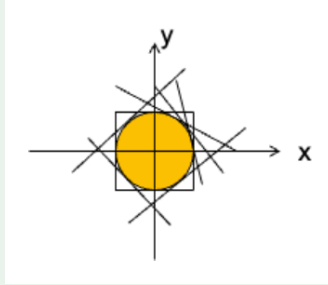


Figura 41: L'immagine illustra l'approssimazione di un cerchio con poliedri convessi.

Mancando un elemento minimo nell'astratto che sovra-approssima il cerchio, non è possibile definire $\alpha(P)$ in modo unico come richiesto dalla definizione di Galois Connection (Definizione 7.2).

7.7 Estensione agli stati

Finora si è parlato di astrazione di valori (es. un intero). Ma le analisi lavorano su stati ($\Sigma : \text{Var} \rightarrow \text{Valori}$).

Proposizione 7.1: Astrazione pointwise

Se esiste una GC tra i valori concreti e i valori astratti, allora l'applicazione puntuale (pointwise) di questa astrazione agli stati forma anch'essa una Connessione di Galois. Questo permette di costruire l'analisi componendo le astrazioni delle singole variabili.

8 Introduzione all'interpretazione astratta

8.1 Semantica astratta sui CFG

Mentre nei capitoli precedenti è stata introdotta la semantica collettrice sul programma sorgente, per l'analisi statica è più conveniente ragionare in termini di CFG. In questa rappresentazione, la semantica dei costrutti di controllo come `while` e `if` è codificata direttamente nella struttura del grafo stesso.

Dato il seguente frammento di codice:

```
1 x = 0;
2 while ( x < 100 )
3   if ( x < 50 )
4     x = x + 2;
5   else
6     x = x + 10;
```

È possibile costruire il seguente CFG (fig. 42):

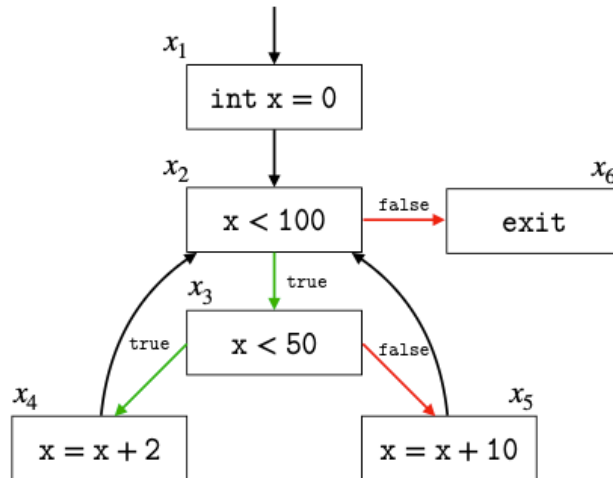


Figura 42: L'immagine illustra il CFG corrispondente al frammento di codice mostrato.

Lo scopo è di definire gli stati di ingresso e uscita per ogni nodo del CFG utilizzando la semantica astratta.

Definizione 8.1: Entry state

Per ogni punto del programma (nodo del CFG) x_i , lo stato di ingresso (*entry*) è definito come il *least upper bound* (Definizione 2.5) degli stati di uscita dei blocchi che puntano ad esso (i predecessori). Per il blocco iniziale, è lo stato di ingresso dato.

Definizione 8.2: Exit state

Per ogni punto del programma (nodo del CFG) x_i , lo stato di uscita (*exit*) è il risultato dell'applicazione della semantica (concreta o astratta) dello statement contenuto nel blocco, applicata allo stato di ingresso.

Poiché la logica di controllo è gestita dagli archi del grafo, è sufficiente esprimere la semantica solo per gli assegnamenti e le espressioni booleane, che fungono da “guardie” per gli archi.

Esempio 8.1: Calcolo degli stati di ingresso

Prendendo in considerazione il CFG in Figura 42, è possibile costruire gli stati di ingresso per ogni blocco come segue:

$$\begin{aligned} x_1 &= I \\ x_2 &= [[x = 0]]x_1 \cup [[x = x + 2]]x_4 \cup [[x = x + 10]]x_5 \\ x_3 &= [[x < 100]]x_2 \\ x_4 &= [[x < 50]]x_3 \\ x_5 &= [[x \geq 50]]x_3 \\ x_6 &= [[x \geq 100]]x_2 \end{aligned}$$

8.2 Sistemi di Equazioni e punto Fisso

Dato un CFG con m nodi si vorrebbe procedere alla risoluzione di un sistema di equazioni.

Definizione 8.3: Sistema di Equazioni Semantiche

Il sistema è definito come:

$$F = \{x_i = f_i(x_1, \dots, x_m) \mid i = 1, \dots, m\}$$

dove ogni x_i rappresenta lo stato (o la proprietà) associata al nodo i .

L'obiettivo è calcolare la soluzione minima del sistema F (Definizione 8.3), che corrisponde al limite di un'iterazione di Kleene sul reticolo del dominio: $\langle \wp(\mathbb{Z}), \subseteq, \emptyset, \mathbb{Z}, \cup, \cap \rangle$.

Nota 8.1: Punto fisso sul dominio concreto

Calcolare il least fixpoint (lfp) sul dominio concreto $(\wp(\mathbb{Z}))$ è spesso indecidibile o non computabile in tempo finito.

8.3 Approssimazione del punto fisso

Dato che molto spesso non è possibile calcolare il punto fisso sul dominio concreto, si introduce un dominio astratto A che approssima il dominio concreto C .

L'obiettivo è calcolare un'approssimazione corretta $\alpha(\text{lfp}(F))$ senza dover calcolare effettivamente $\text{lfp}(F)$.

Per farlo, è necessario definire un sistema di equazioni astratto $F^\#$ e calcolarne il suo punto fisso.

8.3.1 Soundness

Affinché l'analisi sia affidabile, la funzione astratta $f^\#$ deve essere una sound abstraction della funzione concreta f .

Definizione 8.4: Soundness della funzione astratta

Siano C e A due domini legati da una connessione di Galois (α, γ) e siano $f : C \rightarrow C$ una funzione concreta e $f^\# : A \rightarrow A$ una funzione astratta. Allora, $f^\#$ è una sound abstraction di f

$$\forall c \in C. \alpha(f(c)) \sqsubseteq_A f^\#(\alpha(c))$$

Oppure equivalentemente:

$$\forall a \in A. f(\gamma(a)) \sqsubseteq_C \gamma(f^\#(a))$$

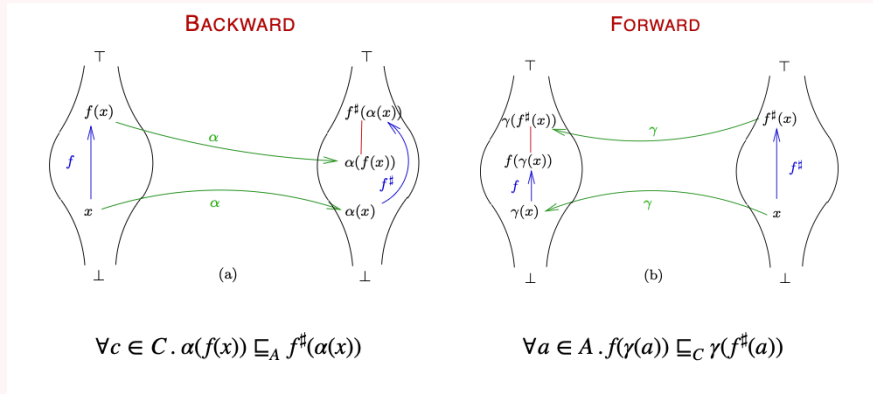


Figura 43: L'immagine illustra la soundness della funzione astratta.

Nota 8.2: Best correct approximation (BCA)

Tra tutte le possibili funzioni astratte corrette, esiste la “migliore” (la più precisa), definita come:

$$f^{bca} = \alpha \circ f \circ \gamma$$

Tuttavia, spesso la BCA non è computabile perché richiede l'applicazione di f e α , operazioni che potrebbero essere troppo complesse.

8.4 Teoremi di trasferimento del punto fisso

Una volta definito il sistema astratto $F^\#$, esistono due teoremi che garantiscono che il suo punto fisso $lfp(F^\#)$ approssimi correttamente il punto fisso concreto $lfp(F)$.

Nota 8.3: Approssimazione e esattezza

$lfp(F^\#)$ è una approssimazione sound di $lfp(F)$ se:

- Esattezza: $\alpha(lfp(F)) = lfp(F^\#)$
- Approssimazione: $\alpha(lfp(F)) \sqsubseteq lfp(F^\#)$

Teorema 8.1: Approssimazione di Kleene

Se $f : C \rightarrow C$ è continua su un reticolo completo concreto $\langle C, \sqsubseteq_C, \perp_C, \top_C, \sqcup_C, \sqcap_C \rangle$ e $f^\# : A \rightarrow A$ è un'astrazione sound di f su un dominio astratto $\langle A, \sqsubseteq_A, \perp_A, \top_A, \sqcup_A, \sqcap_A \rangle$, e se la sequenza $\{f_i^\#(\perp_A) \mid i \in \mathbb{N}\}$ ha un limite $x \in A$, allora è una approssimazione sound di $lfp(f) \sqsubseteq_C \gamma(x)$. In particolare, ad ogni passo i , l'iterazione astratta $f_i^\#(\perp_A)$ è una approssimazione sound di $f_i(\perp_C)$.

Nota 8.4: Approssimazione di Kleene

Questo teorema garantisce che l'iterazione di Kleene sul dominio astratto fornisce una approssimazione sound del punto fisso concreto. Tuttavia, non garantisce che l'approssimazione sia la migliore possibile (neanche alla convergenza).

Teorema 8.2: Approssimazione di Tarski

Dato un reticolo completo $\langle C, \sqsubseteq_C, \perp_C, \top_C, \sqcup_C, \sqcap_C \rangle$ e una funzione monotona $f : C \rightarrow C$, e una sua astrazione sound $f^\# : A \rightarrow A$ in un dominio astratto $\langle A, \sqsubseteq_A, \perp_A, \top_A, \sqcup_A, \sqcap_A \rangle$, allora qualsiasi **post-fixpoint** a di $f^\#$ (ovvero tale che $f^\#(a) \sqsubseteq_A a$) è un'approssimazione sound del punto fisso concreto $lfp(f)$, ovvero:

$$lfp(f) \sqsubseteq_C \gamma(a)$$

Nota 8.5: Approssimazione di Tarski

Il teorema di approssimazione di Tarski garantisce che qualsiasi post-fixpoint della funzione astratta è una approssimazione sound del punto fisso concreto.

9 Ottimizzazione dell'Analisi: l'operatore split

Questa sezione approfondisce una tecnica per accelerare l'analisi statica riducendo la ridondanza nelle operazioni di filtraggio sui domini astratti

9.1 Filter operator e problemi associati

Nell'interpretazione astratta classica, la gestione del flusso di controllo condizionale (es. `if (cond) ...else ...`) viene gestita tramite l'operatore di **filtro**

Definizione 9.1: Filter operator

L'operatore di filtro è definito come una funzione:

$$filter : \mathbb{A} \times Pred \rightarrow \mathbb{A}$$

dove $\mathbb{A}$ è il dominio astratto e $Pred$ è un predicato (condizione).

Nota 9.1: Utilizzo dell'operatore di filtro

L'operatore di filtro (Definizione 9.1) viene utilizzato per modellare il flusso di controllo, raffinando lo stato astratto in base alla condizione incontrata (es. $i < 50$ nel ramo *then* e $i \geq 50$ nel ramo *else*).

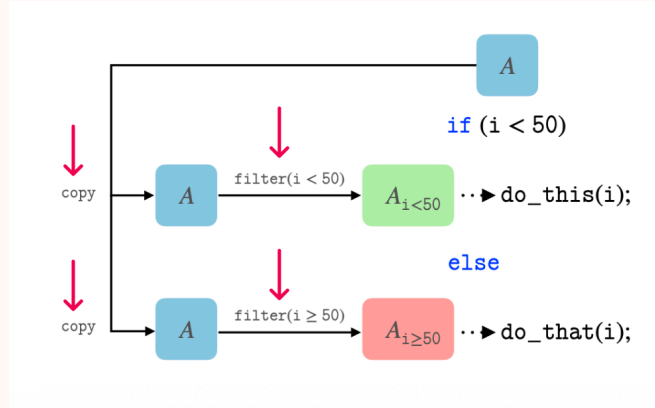


Figura 44: L'immagine illustra l'uso dell'operatore di filtro per gestire un ramo condizionale in un CFG. Lo stato astratto A viene filtrato in due stati distinti in base alla condizione c e alla sua negazione $\neg c$.

L'approccio tradizionale presenta delle inefficienze strutturali:

1. **Duplicazione del lavoro:** Per gestire un ramo condizionale, è necessario invocare l'operatore due volte, una per la condizione positiva ($filter(A, c)$) e una per la negazione ($filter(A, \neg c)$).

2. **Overhead di memoria:** Spesso richiede la copia preliminare dello stato astratto A per preservarlo per la seconda invocazione.

9.2 Split operator

Per risolvere le inefficienze del filtro standard, viene introdotto l'operatore **split**.

Definizione 9.2: Split operator

L'operatore split combina le due operazioni di filtraggio in un'unica invocazione atomica:

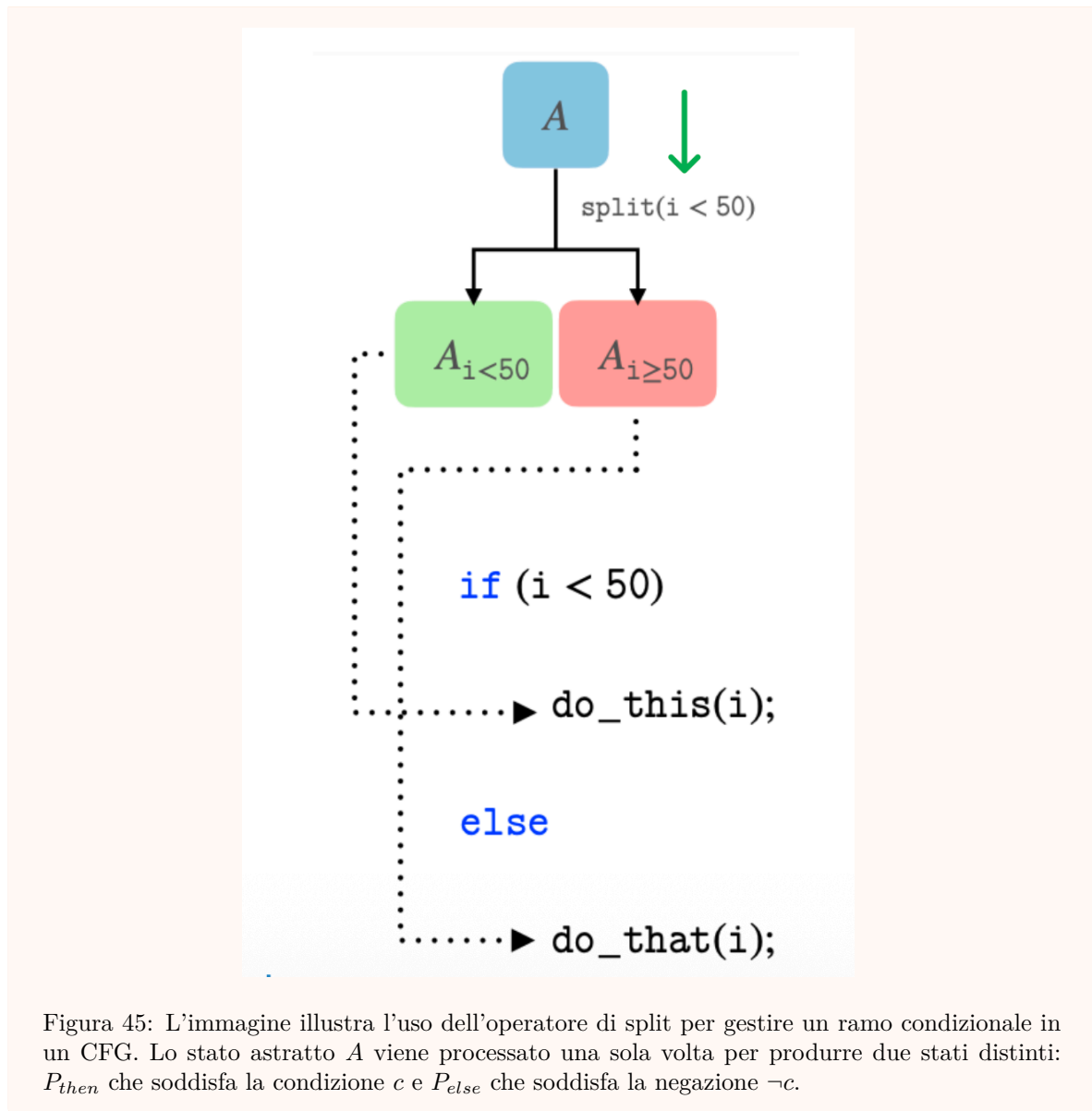
$$split : \mathbb{A} \times Pred \rightarrow \mathbb{A} \times \mathbb{A}$$

L'operatore prende in input uno stato astratto e un predicato, e restituisce una coppia di stati astratti. Uno soddisfa il predicato (P_{then}) e uno che soddisfa il suo complemento (P_{else}).

Nota 9.2: Vantaggi del Split

L'utilizzo dell'operatore split permette di:

- Eliminare il lavoro replicato processando il vincolo e il suo opposto contemporaneamente.
- Ridurre gli overhead evitando copie esplicite dello stato astratto prima dell'operazione, delegando la gestione della memoria alla libreria del dominio sottostante.



9.3 Implementazione e Integrazione

L'operatore è stato implementato all'interno della libreria per poliedri PPLite e integrato nel framework di analisi statica Crab.

9.3.1 Modifiche al CFG

L'integrazione richiede una modifica strutturale al Control Flow Graph (CFG) analizzato. Mentre un CFG standard utilizza archi annotati con *assume* (o guardie) uscenti da un blocco condizionale, la versione ottimizzata introduce uno statement esplicito *split*.

- **Standard CFG:** Archi separati con *assume(cond)* e *assume($\neg$ cond)*.
- **Split CFG:** Un singolo nodo contenente l'istruzione *split(cond)*, che propaga internamente i due stati risultanti ai successori.

Esempio 9.1: Modifica del CFG con Split

Prendendo in considerazione il seguente frammento di codice:

```
void foo() {
    int i = 0;
    while (i < 50) {
        i = bar(i);
    }
}
```

Da questo si possono ottenere i CFG rappresentati in 46.

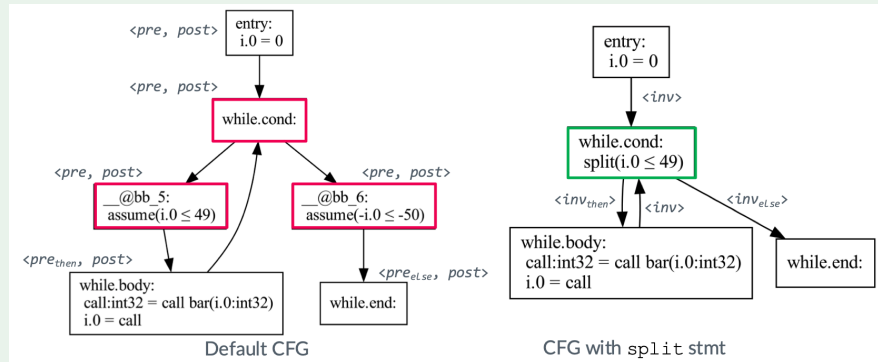


Figura 46: L'immagine illustra la differenza tra un CFG standard (a sinistra) e un CFG ottimizzato con l'operatore *split* (a destra) per il frammento di codice fornito. Nel CFG standard, il nodo condizionale genera due archi distinti con *assume*($i < 50$) e *assume*($i \geq 50$). Nel CFG con *split*, un singolo nodo esegue l'operazione *split*($i < 50$), propagando i due stati risultanti ai successori.

9.4 Valutazione Sperimentale

L'approccio è stato valutato su benchmark reali provenienti da progetti differenti. I test hanno mostrato miglioramenti significativi in termini di tempo di analisi (47) e utilizzo della memoria (48), senza compromettere la precisione dei risultati.

(abridged) test name	without splits		with splits	time ratio
	PPLite	ELINA	PPLite	
adpcm	108.2	(★) 1.6	105.9	1.02
prog9000	31.8	241.1	42.0	0.76
nsichneu	30.9	40.8	21.1	1.46
decompress	5.3	5.5	2.3	2.30
filter	2.5	2.3	2.5	1.00
mmc-host	487.1	435.1	412.9	1.18
firewire	98.7	(★) 92.0	22.7	4.35
hwmon-abituguru3	87.5	91.0	52.8	1.66
media-usb-tm6000	23.2	22.3	12.4	1.87
media-pci-ttpci	12.3	12.9	10.8	1.14
power-bq2415x	10.1	10.9	11.6	0.87
hid-usb	8.6	10.1	6.0	1.43

Figura 47: L'immagine illustra i risultati sperimentali in termini di tempo.

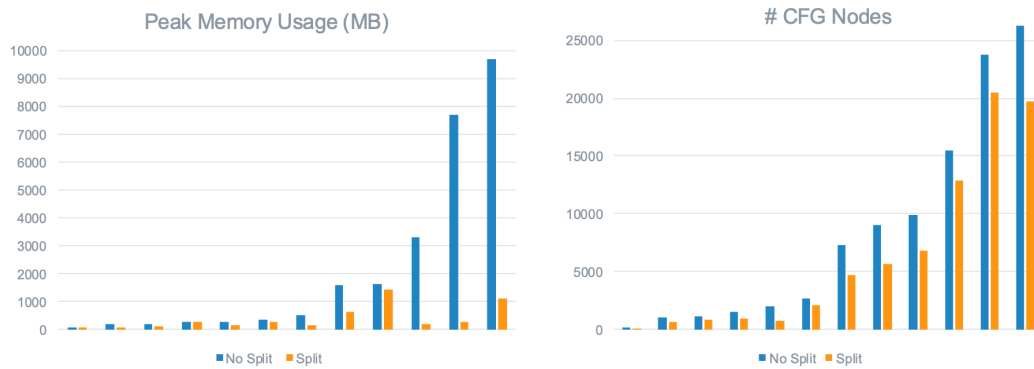


Figura 48: L'immagine illustra i risultati sperimentali in termini di utilizzo della memoria e numero di nodi del CFG.

10 Ottimizzazione dell'Analisi Statica: Oracoli per Variabili Non Vincolate

10.1 Contesto e abstract compilation

L'Interpretazione Astratta è una tecnica potente, ma può introdurre inefficienze significative, specialmente quando si utilizzano domini astratti relazionali complessi. Le soluzioni canoniche per migliorare le performance solitamente prevedono la scelta di un dominio astratto diverso o di operatori astratti differenti.

Tuttavia, esiste un approccio alternativo chiamato **Abstract compilation**.

Definizione 10.1: Abstract compilation

Le tecniche di Abstract compilation consistono nel riscrivere la rappresentazione intermedia del programma adottata dall'analizzatore. L'obiettivo è ottimizzare l'esecuzione astratta modificando il codice su cui l'analisi opererà, cercando un trade-off (compromesso) tra accuratezza ed efficienza.

L'approccio proposto in questa lezione si basa sull'idea che l'efficienza del processo di valutazione può essere migliorata propagando non solo l'informazione nota, ma anche l'informazione "sconosciuta".

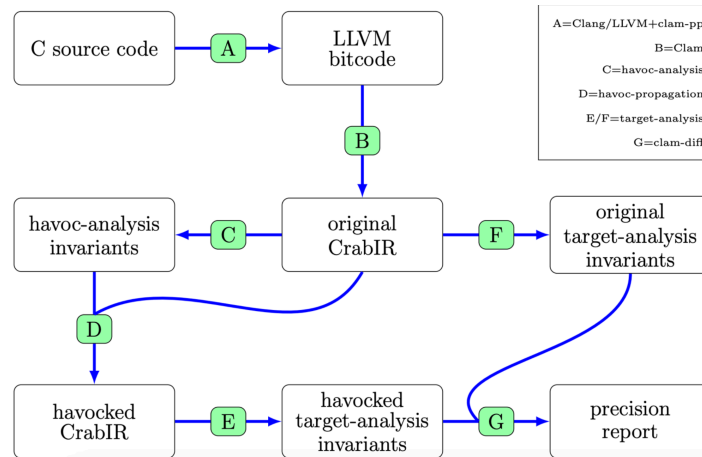


Figura 49: L'immagine illustra la pipeline dell'analisi statica con oracoli per variabili non vincolate.

10.2 Likely unconstrained variables

Il concetto chiave è l'identificazione delle likely unconstrained variables (LUV), ovvero variabili che molto probabilmente non avranno vincoli (saranno $\top$, o $[-\infty, +\infty]$) in un determinato punto del programma.

Esempio 10.1: Propagazione dell'incertezza

Considerando l'espressione $x = y + expr$. Sapendo che y è "unconstrained" (cioè $y \rightarrow [-\infty, +\infty]$) e $expr$ ha un valore limitato (es. $[1, 12]$), allora anche il risultato x sarà necessariamente $[-\infty, +\infty]$.

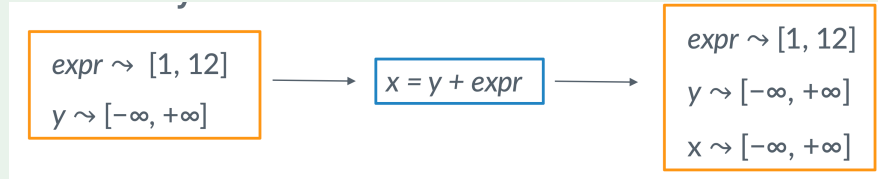


Figura 50: L'immagine illustra la propagazione dell'incertezza attraverso un'operazione aritmetica.

Se non si ha alcuna informazione su y , è probabile che non si avrà alcuna informazione sull'intera espressione. Di conseguenza, c'è poco incentivo a fornire una valutazione astratta precisa (e costosa) per quell'espressione, poiché il risultato sarà comunque $\top$.

L'obiettivo è identificare queste variabili e propagare questa "mancanza di informazione" per semplificare l'analisi.

10.3 La Pipeline di propagazione LUV

L'approccio si divide in passaggi distinti, che fungono da pre-analisi leggera prima dell'analisi target costosa:

1. **Dataflow analysis:** Identificazione delle variabili LUV tramite euristiche (Oracoli).
2. **Propagation:** Diffusione dell'informazione nel CFG (Esistenziale o Universale).
3. **Program transformation:** Riscrittura del CFG sostituendo le istruzioni complesse con assegniamenti "havoc".

10.3.1 Oracoli e euristiche

Poiché determinare esattamente se una variabile sarà $\top$ è costoso quanto l'analisi stessa, si utilizzano degli **oracoli** euristici.

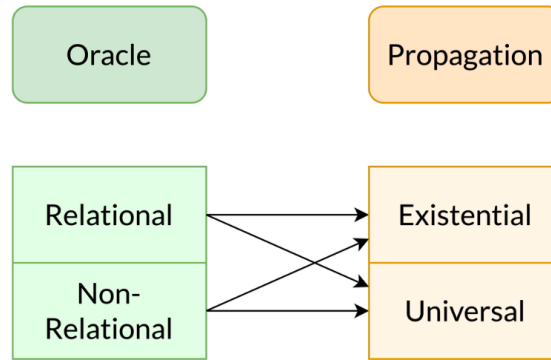


Figura 51: L'immagine illustra le diverse tipologie di oracoli.

Esistono varianti relazionali e non-relazionali.

Definizione 10.2: Oracolo non-relazionale

Un oracolo non-relazionale decide se una variabile è LUV basandosi solo sulle proprietà locali dell'assegnamento. Ad esempio, per l'operazione $x \leftarrow y \text{ op } z$:

- Se y è LUV o z è LUV, allora x diventa LUV.
- L'oracolo è aggressivo: tende a classificare più variabili come LUV.

Definizione 10.3: Oracolo relazionale

Un oracolo relazionale cerca di preservare le relazioni tra variabili. Ad esempio, per $x \leftarrow y \text{ op } z$:

- Se $op \in \{+, -\}$ e le variabili sono coinvolte, l'oracolo potrebbe decidere di non marcare x come LUV immediatamente, per preservare potenziali vincoli relazionali.
- Questo oracolo è più conservativo (meno falsi positivi, ma meno ottimizzazione).

Nota 10.1: Falsi positivi e negativi

- **Falso positivo (Perdita di precisione):** L'oracolo dice che x è LUV, ma l'analisi concreta avrebbe trovato un vincolo (es. $x \in [0, 0]$). Questo porta a una perdita di precisione nell'analisi finale.
- **Falso negativo (Mancata ottimizzazione):** L'oracolo dice che x è vincolata, ma in realtà è $[-\infty, +\infty]$. L'analisi rimane corretta ma non viene ottimizzata.

Indipendentemente dall'euristica, l'analisi finale è sempre **sound** (corretta).

10.3.2 Strategie di Propagazione

Una volta identificate le LUV localmente, l'informazione deve essere propagata attraverso i blocchi base del Control Flow Graph (CFG). I tipi principali di propagazione sono:

- **Propagazione esistenziale** ($\exists$): Una variabile è nel set LUV all'inizio di un blocco se è LUV lungo *almeno uno* degli archi entranti.
- **Propagazione universale** ($\forall$): Una variabile è nel set LUV all'inizio di un blocco se è LUV lungo *tutti* gli archi entranti.

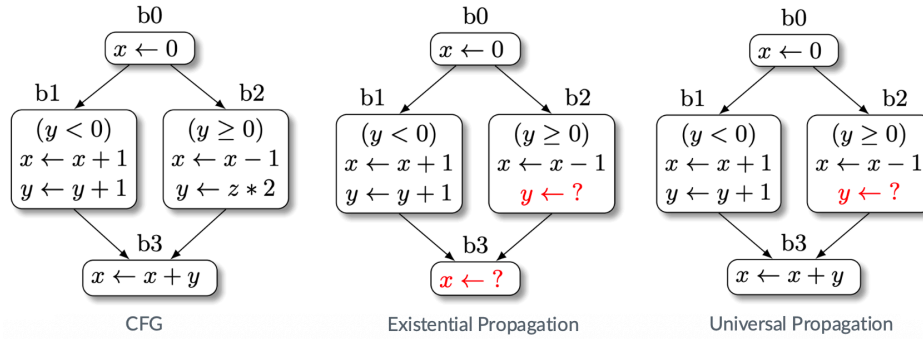


Figura 52: L'immagine illustra un esempio di propagazione esistenziale e universale.

10.4 Trasformazione del programma (Havoc)

L'ultimo passo è la trasformazione del codice intermedio (es. CrabIR). L'algoritmo prende in input il CFG e la mappa delle variabili LUV post-analisi (LU_{post}). Per ogni istruzione $x \leftarrow expr$:

- Se $x \in LU_{post}$ del blocco corrente, l'istruzione viene sostituita con $x \leftarrow ?$ (dove ? rappresenta un valore non deterministico o "havoc").

Questo rimuove il carico computazionale di valutare $expr$ nell'analisi successiva.

10.5 Valutazione sperimentale

L'approccio è stato implementato e testato su vari benchmark.

L'utilizzo di oracoli per variabili non vincolate ha dimostrato:

- **Speedup significativo:** Velocizzazione media di 2.61x utilizzando il dominio dei poliedri, con picchi fino a 8.23x.
- **Precisione:** La perdita di precisione è limitata. Con i poliedri si mantiene circa il 90% dei vincoli originali (invarianti).
- Le varianti non-relazionali degli oracoli tendono ad essere più aggressive e a garantire maggiori miglioramenti di efficienza rispetto alla scelta tra propagazione esistenziale o universale.

11 Interpretazione Astratta in LiSA

11.1 Cos'è LiSA

Definizione 11.1: LiSA (Library for Static Analysis)

LiSA è una libreria open-source scritta in Java, sviluppata e mantenuta dal gruppo di ricerca SSV dell'Università Ca' Foscari di Venezia. Il suo scopo è fornire un framework modulare e estensibile per la costruzione di analizzatori statici.

I principi fondamentali su cui si basa LiSA sono:

- Un programma in LiSA è rappresentato come un insieme di control flow graphs (CFGs), più eventuali informazioni ausiliarie.
- I CFG codificano il flusso di controllo nella loro struttura, astruendo dai costrutti sintattici del linguaggio sorgente.
- I nodi del CFG sono specifici del linguaggio, ma vengono riscritti in **espressioni simboliche** generiche.

Nota 11.1: Espressioni simboliche

La riscrittura dei CFG in espressioni simboliche generiche permette di separare la sintassi dalla semantica, consentendo ai domini astratti di operare in modo agnostico rispetto al linguaggio.

11.2 Architettura di LiSA

L'architettura di LiSA è progettata per essere altamente modulare, separando i componenti dell'analisi per massimizzare la riusabilità.

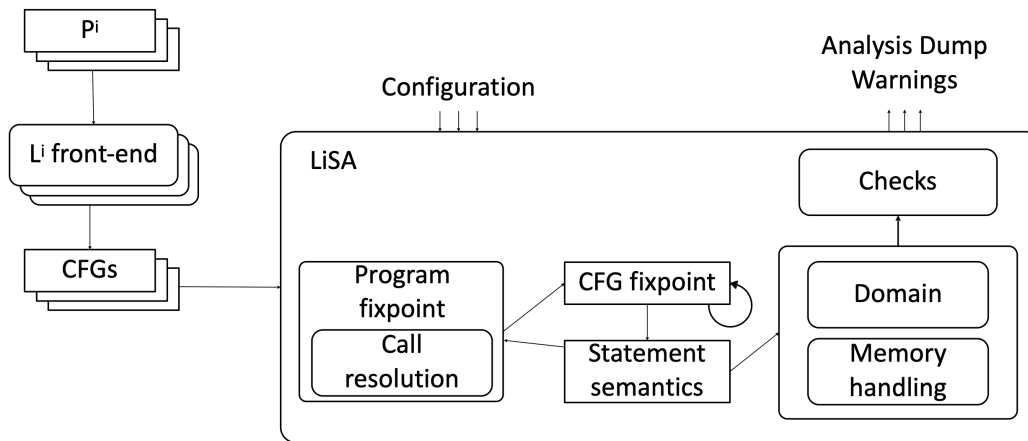


Figura 53: L'immagine illustra la visione d'insieme dell'architettura di LiSA.

I componenti principali, come mostrato in Figura 53, sono:

- **Li front-end**: Traduce il programma sorgente (P_i) in una collezione di CFG.
- **LiSA “core”**: È il cuore dell’analizzatore e include:
 - **CFG fixpoint**: L’engine che implementa l’algoritmo di punto fisso basato su worklist per calcolare la semantica astratta.
 - **Statement semantics**: Logica per tradurre i nodi del CFG in espressioni simboliche.
 - **Domain**: Il dominio astratto che definisce le proprietà da tracciare (es. intervalli, segni).
 - **Memory handling**: Componente che modella la memoria (heap).
 - **Program fixpoint**: Gestisce l’analisi interprocedurale e la risoluzione delle chiamate (call resolution).
- **Checks**: Componenti che verificano la presenza di specifici bug o proprietà sui risultati dell’analisi.
- **Analysis dump**: Produce report e warning per l’utente.

11.3 Calcolo del punto fisso sul CFG

L’analisi intra-procedurale in LiSA si basa su un calcolo di punto fisso eseguito direttamente sul CFG.

Definizione 11.2: Algoritmo di punto fisso del CFG

L’algoritmo è basato su un approccio worklist per raggiungere la stabilità. Lo pseudocodice è il seguente:

```

result = {}
ws = {entry_node}
while (ws is not empty) do
  st = ws.pop()
  in_st = LUB { out_m | m in preds(st) }
  out_st = semantics(st, in_st)
  if (not out_st <= result[st]) then
    if (widening_threshold_reached) then
      result[st] = result[st] WIDEN out_st
    else
      result[st] = result[st] LUB out_st
    end if
    ws.push(successors(st))
  end if
end while

```

Nota 11.2: Componenti dell'algoritmo

L'algoritmo si basa su operazioni definite dal dominio astratto, che deve essere un reticolo:

- **LUB (least upper bound):** Unisce l'informazione proveniente da più cammini di controllo.
- **Partial order:** Verifica se un nuovo stato aggiunge informazione rispetto a quello già calcolato, per determinare se la computazione si è stabilizzata.
- **Widening:** Operatore di allargamento (prossimi capitoli) che garantisce la terminazione dell'analisi su reticoli di altezza infinita, sacrificando la precisione per forzare la convergenza.

Gli stati calcolati devono quindi essere elementi di un reticolo (Definizione 2.9).

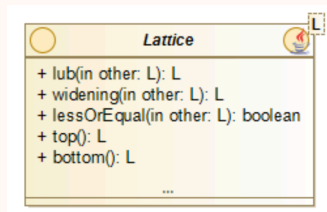


Figura 54: L'immagine illustra le operazioni principali su un reticolo.

11.4 Semantica degli statement

In LiSA, la semantica di uno statement astrae dalle operazioni concrete e si concentra su tre funzionalità principali che un dominio deve supportare.

Definizione 11.3: SemanticDomain

Uno stato astratto in LiSA deve implementare l'interfaccia **SemanticDomain**, che fornisce metodi per:

- **assign(id, expr, ...):** Gestisce l'assegnamento di un'espressione a una variabile.
- **smallStepSemantics(expr, ...):** Valuta un'espressione simbolica.
- **satisfies(expr, ...):** Verifica se un'espressione è soddisfatta dallo stato astratto.
- **assume(expr, ...):** Raffina lo stato astratto assumendo che un'espressione sia vera.

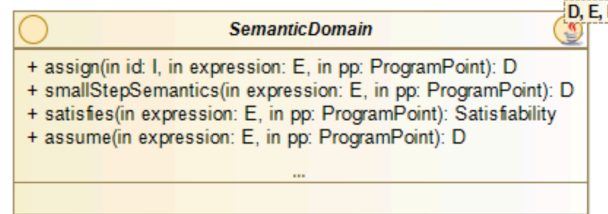


Figura 55: L'immagine illustra le operazioni principali sul SemanticDomain.

Esempio 11.1: Semantica di un condizionale

La semantica di un'istruzione condizionale `if (cond) then ...` viene gestita come segue:

```
// Calcola lo stato per il ramo "then"
tbranch = domain.assume(cond).smallStepSemantics(left)
// Calcola lo stato per il ramo "else"
fbranch = domain.assume(cond.negate()).smallStepSemantics(right)

// Verifica la soddisfacibilità per potare i rami
if domain.satisfies(cond) == SATISFIED:
    return tbranch
else if domain.satisfies(cond) == NOT_SATISFIED:
    return fbranch
else: // Se non so, unisco i risultati
    return tbranch.lub(fbranch)
```

11.5 Gestione della memoria e stato astratto

Per garantire la modularità, LiSA separa la gestione dei valori dalla gestione della memoria.

11.5.1 Gestione della memoria dinamica

Nota 11.3: Indipendenza dalla memoria

I domini astratti (es. intervalli, segni) non dovrebbero preoccuparsi di come la memoria è organizzata. L'idea è trattare ogni locazione di memoria (es. un campo di un oggetto) come una "variabile fittizia" (fake variable).

Esempio 11.2: Accesso a un campo di un oggetto

Considerando il seguente frammento di codice che alloca due oggetti:

```
class A {
```

```

    int f, g;
}
x = new A();
y = new A();

```

L'Heap Domain mappa ogni oggetto e i suoi campi a una locazione di memoria concreta, che viene trattata come una variabile fittizia. Come mostrato in 56, l'oggetto puntato da x si trova alla locazione L_1 , mentre quello puntato da y si trova in L_2 . Di conseguenza, l'accesso al campo f di x ($x.f$) viene risolto nella variabile fittizia $L_{1.1}$, mentre $y.f$ viene risolto in $L_{2.1}$. Sarà poi compito del Value Domain associare un valore astratto a queste variabili fittizie (es. $L_{1.1} \rightarrow [5, 5]$).

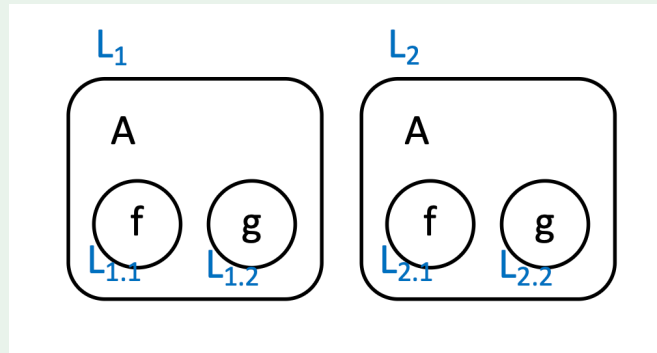


Figura 56: L'immagine illustra la modellazione della memoria dinamica. Ogni oggetto e campo viene mappato a una locazione di memoria unica.

11.5.2 Lo stato astratto

Lo stato astratto (57) in LiSA è tipicamente composto da due parti:

- **Heap domain:** Si occupa di modellare la memoria. La sua responsabilità è risolvere un'espressione che accede alla memoria (es. $x.f$) nella corrispondente variabile fittizia che rappresenta quella locazione.
- **Value domain:** Mappa le variabili (sia quelle dello stack che quelle fittizie dello heap) ai loro valori astratti (es. intervalli, segni).

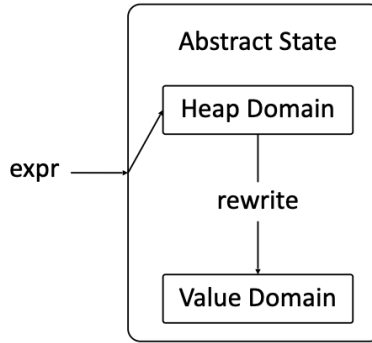


Figura 57: L'immagine illustra la composizione dello stato astratto in LiSA.

A seconda della precisione richiesta, l'heap domain può modellare le locazioni di memoria in modi diversi (es. context-sensitive, context-insensitive), influenzando il trade-off tra precisione e performance.

11.6 Analisi interprocedurale

L'analisi interprocedurale gestisce le chiamate a funzione, orchestrando le analisi intra-procedurali dei singoli CFG.

Definizione 11.4: Analisi Interprocedurale

Questo componente è responsabile di:

- Analizzare i CFG in isolamento.
- Iniziare l'analisi dal “main” e seguire le chiamate a funzione.
- Utilizzare un **call graph** per risolvere dinamicamente i target di una chiamata (es. in presenza di polimorfismo).

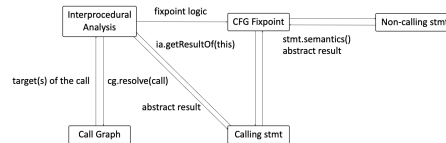


Figura 58: L'immagine illustra l'analisi interprocedurale in LiSA.

11.7 Domini astratti: relazionali e non-relazionali

I domini di valori si dividono in due categorie principali:

- **Domini non-relazionali:** Tracciano i valori di ogni variabile in modo indipendente.
- **Domini relazionali:** Tracciano le relazioni tra i valori di più variabili.

11.7.1 Domini non-relazionali

Definizione 11.5: Dominio non-relazionale

Un dominio non-relazionale traccia i valori di ogni variabile in modo indipendente, tramite una mappa da identificatori a valori astratti.

In questo tipo di domini la logica di valutazione è specifica per il dominio.

Esempio 11.3: Esempi di domini non-relazionali

Alcuni esempi di domini non-relazionali sono:

- Constant Propagation ($x \rightarrow 3$).
- Intervals ($x \rightarrow [2, 10]$).

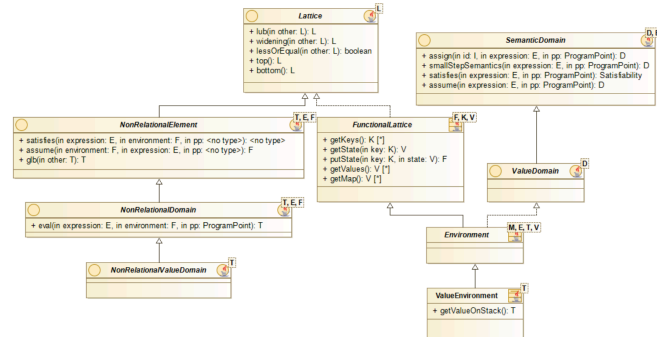


Figura 59: L'immagine illustra le classi di domini non-relazionali in LiSA.

11.7.2 Domini relazionali

Definizione 11.6: Dominio relazionale

Un dominio relazionale traccia le relazioni tra i valori di più variabili.

- **Esempi:** Upper bounds ($x > y$), Linear inequalities ($x + y > 10, z < 0$).

Esempio 11.4: Esempi di domini relazionali

Alcuni esempi di domini relazionali sono:

- Upper bounds ($x > y$).
- Linear inequalities ($x + y > 10, z < 0$).

12 Domini numerici e interpretazione astratta

12.1 Il problema generale

Nota 12.1: Problemi di affidabilità del software

L'hardware è diventato milioni di volte più potente rispetto a 25 anni fa, e la dimensione dei programmi è cresciuta in modo simile, raggiungendo decine di milioni di righe di codice. Questi programmi devono essere progettati, sviluppati e mantenuti per lunghi periodi (fino a 20 anni e più) a costi ragionevoli.

I team di sviluppo e manutenzione non assistiti non possono far fronte a questa esplosione di dimensioni e complessità. Molti software presentano un numero di bug a volte difficilmente sopportabile anche nelle applicazioni da ufficio. Nessuna applicazione critica per la sicurezza può tollerare questo tasso di fallimento.

Il problema dell'affidabilità del software è uno dei problemi più importanti che l'informatica deve affrontare. Questo giustifica il crescente interesse per strumenti meccanici che aiutino il programmatore a ragionare sui programmi.

12.2 Esempi di problemi di verifica

Esempio 12.1: Disastro di Ariane 5

Il disastro del volo 501 di Ariane 5 è stato causato da un segmento di programma che convertiva un numero in virgola mobile in un intero a 16 bit con segno. Il programma è stato eseguito con un valore di input al di fuori dell'intervallo rappresentabile da un intero a 16 bit con segno. Questo errore di runtime (overflow), si è verificato sia nei computer attivi che in quelli di backup quasi contemporaneamente, ed entrambi i computer si sono spenti. Ciò ha comportato la perdita totale del controllo dell'assetto. L'Ariane 5 è diventato incontrollabile e le forze aerodinamiche hanno distrutto il veicolo.

Esempio 12.2: Disastri storici

- Il Mars Climate Orbiter si è disintegrato nell'atmosfera marziana nel 1999 dopo aver mancato l'inserimento in orbita a causa di calcoli incoerenti sulle unità di misura.
- Nello stesso anno, si sospetta che il Mars Polar Lander si sia schiantato su Marte all'atterraggio perché un flag software non è stato resettato correttamente.
- Nella missione dimostrativa tecnologica Mars Pathfinder (MPF) del 1997, un giorno di esplorazione è andato perso quando i team di supporto a terra sono stati costretti a riavviare il sistema durante il download dei dati scientifici.
- La missione Mars Exploration Rover (MER) della NASA del 2003 comprende due rover. Con un costo di 400 milioni di dollari per ogni rover, un errore di codifica che spegne un rover durante la notte sarebbe a tutti gli effetti un errore da 4,4 milioni di dollari, oltre a una perdita di tempo prezioso di esplorazione sul pianeta.

Esempio 12.3: È ben definito $x/(x - y)$?

Molte cose possono andare storte:

- x e/o y potrebbero non essere inizializzati;
- $x - y$ potrebbe andare in overflow;
- x e y potrebbero essere uguali (o $x - y$ potrebbe andare in underflow): divisione per 0;
- $x/(x - y)$ potrebbe andare in overflow (o underflow).

12.3 Metodi formali di verifica dei programmi

Definizione 12.1: Scopo della verifica formale

Lo scopo della verifica formale è dimostrare meccanicamente che tutte le possibili esecuzioni di un programma sono corrette in tutti gli ambienti di esecuzione specificati.

Esistono diversi metodi di verifica:

- Metodi deduttivi.
- Model checking.
- Tipizzazione dei programmi.
- Analisi statica.

Nota 12.2: L'indecidibilità sui metodi di verifica

A causa dell'indecidibilità della verifica dei programmi, tutti i metodi sono parziali o incompleti e ricorrono a una qualche forma di approssimazione.

12.4 Interpretazione astratta

Definizione 12.2: Interpretazione astratta

L'interpretazione astratta è il framework giusto per lavorare con il concetto di approssimazione corretta. È una teoria per approssimare insiemi e operazioni su insiemi come considerato nella teoria degli insiemi (o delle categorie), comprese le definizioni induttive. È una teoria dell'approssimazione del comportamento dei sistemi discreti dinamici. La computazione avviene su un dominio di proprietà astratte: il dominio astratto, utilizzando operazioni astratte che sono approssimazioni corrette delle operazioni concrete. La correttezza segue dal progetto.

Nota 12.3: Livelli di approssimazione

L'astrazione (approssimazione) può essere abbastanza grossolana da essere calcolabile finitamente, ma abbastanza precisa da essere praticamente utile.

12.5 Domini astratti numerici

Definizione 12.3: Domini astratti numerici

I domini astratti numerici sono utilizzati per rappresentare proprietà numeriche delle variabili di un programma. Alcuni esempi di domini astratti numerici sono:

- Segni
- Bounding Boxes
- Congruenze semplici
- Differenze limitate
- Ottagoni
- Poliedri convessi
- Z-Polyhedra
- Congruenze relazionali
- Coni polinomiali

12.6 Un assaggio di semantica concreta e astratta

Nota 12.4: Semantica Concreta vs. Astratta

La semantica concreta descrive l'esatto comportamento di un programma, mentre la semantica astratta ne descrive un comportamento approssimato. La semantica concreta opera su un dominio concreto (es. $\wp(\mathbb{R}^2)$), mentre la semantica astratta opera su un dominio astratto (es. poliedri convessi).

Esempio 12.4: Semantica concreta

Considerando il seguente programma:

```
x := 0; y := 0;
while x <= 100 do
  read(b);
  if b then x := x+2
  else x := x+1; y := y+1;
endif
```

endwhile

La semantica concreta calcola l'insieme esatto di tutti i possibili valori di (x, y) in ogni punto del programma.

Esempio 12.5: Semantica astratta

La semantica astratta calcola un'approssimazione dell'insieme di tutti i possibili valori di (x, y) in ogni punto del programma. Per garantire la terminazione, vengono utilizzati operatori di widening per accelerare la convergenza del punto fisso.

13 Widening

Nei domini astratti con catene ascendenti infinite, l'iterazione standard per il calcolo del punto fisso potrebbe non terminare. L'operatore di widening (∇) è una tecnica fondamentale per garantire la convergenza dell'analisi in un numero finito di passi, forzando una stabilizzazione della sequenza di iterati.

13.1 Definizione dell'operatore di widening

Definizione 13.1: Operatore di widening

Un operatore binario $\nabla : A \times A \rightarrow A$ è un operatore di widening su un dominio astratto A (che è un join-semilattice) se soddisfa due proprietà fondamentali:

- **Upper bound (Sovra-approssimazione):**

$$\forall x, y \in A : x \sqsubseteq x \nabla y \quad \text{e} \quad y \sqsubseteq x \nabla y$$

Il risultato deve essere un upper bound di entrambi gli operandi (quindi anche della loro unione $x \sqcup y$).

- **Convergenza:** Per ogni catena ascendente $(y_i)_{i \in \mathbb{N}}$ in A , la sequenza $(x_i)_{i \in \mathbb{N}}$ definita come:

$$x_0 = y_0, \quad x_{i+1} = x_i \nabla y_{i+1}$$

si stabilizza in un tempo finito. Ovvero $\exists k \geq 0$ tale che $x_{k+1} = x_k$.

Esempio 13.1: Widening su intervalli

Nel dominio degli intervalli, l'operatore di widening (Definizione 13.1) è progettato per proiettare all'infinito gli estremi che risultano instabili (che crescono) tra un'iterazione e l'altra.

Dati due intervalli $I_1 = [a, b]$ e $I_2 = [c, d]$ tali che $I_1 \sqsubseteq I_2$, il widening è definito come:

$$[a, b] \nabla [c, d] = \left[\begin{cases} a & \text{se } a \leq c \\ -\infty & \text{altrimenti} \end{cases}, \begin{cases} b & \text{se } b \geq d \\ +\infty & \text{altrimenti} \end{cases} \right]$$

Se un estremo è stabile (es. $a \leq c$ per il limite inferiore), viene mantenuto. Se invece l'estremo si sta "allargando" ($a > c$), il widening lo forza immediatamente a $-\infty$ (o $+\infty$ per il limite superiore) per garantire che smetta di cambiare.

Proprietà:

- **Non commutativo:** $[2, 3] \nabla [1, 4] = [-\infty, +\infty] \neq [1, 4] \nabla [2, 3]$.
- **Non monotono:** Può accadere che pur avendo input più precisi, il risultato del widening sia meno preciso.

13.2 Strategie di applicazione

L'applicazione indiscriminata del widening a tutte le equazioni porterebbe a una grossolana sovrapprossimazione. Secondo Bourdoncle, è sufficiente applicare il widening solo in un insieme di punti che interrompono ogni ciclo nel grafo delle dipendenze (feedback vertex set), tipicamente le intestazioni dei cicli (loop headers).

Il sistema di equazioni astratto diventa quindi:

$$x_i = \begin{cases} x_i \nabla f_i^\#(x_1, \dots, x_m) & \text{se } i \in WP \text{ (Widening Points)} \\ x_i \sqcup f_i^\#(x_1, \dots, x_m) & \text{altrimenti} \end{cases}$$

14 Narrowing

L'utilizzo del widening garantisce la convergenza ma introduce una perdita di precisione significativa (spesso introducendo valori $\pm\infty$). Il risultato ottenuto è un post-punto fisso ($postfp(f)$), tale che $F^\#(x) \sqsubseteq x$. L'operatore di **narrowing** (Δ) permette di raffinare questo risultato recuperando precisione.

Definizione 14.1: Operatore di Narrowing

Un operatore binario $\Delta : A \times A \rightarrow A$ è un operatore di narrowing se soddisfa le seguenti proprietà:

- **Raffinamento limitato:**

$$\forall x, y \in A : x \sqcap y \sqsubseteq x \Delta y \sqsubseteq x$$

Il risultato deve essere contenuto nel primo operando x (migliorando la precisione) ma deve comunque contenere l'intersezione reale $x \sqcap y$ (per mantenere la soundness).

- **Convergenza:** Per ogni sequenza $(y_i)_{i \in \mathbb{N}}$, la catena $(x_i)_{i \in \mathbb{N}}$ definita come:

$$x_0 = y_0, \quad x_{i+1} = x_i \Delta y_{i+1}$$

si stabilizza in un tempo finito.

Esempio 14.1: Narrowing su Intervalli

Nel dominio degli intervalli, il narrowing permette di recuperare estremi finiti partendo da estremi infiniti generati dal widening, ma solo se il nuovo valore calcolato è un miglioramento valido.

$$[a, b] \Delta [c, d] = \left[\begin{cases} c & \text{se } a = -\infty \\ a & \text{altrimenti} \end{cases}, \begin{cases} d & \text{se } b = +\infty \\ b & \text{altrimenti} \end{cases} \right]$$

La logica è la seguente:

- Se il vecchio estremo era infinito ($-\infty$ o $+\infty$) e il nuovo valore proposto è finito, accettiamo il nuovo valore (abbiamo recuperato precisione).
- Se il vecchio estremo era già finito, lo manteniamo inalterato anche se il nuovo valore è “migliore” (più stretto). Questo è cruciale per garantire la terminazione ed evitare cicli infiniti di raffinamento.

14.1 Fasi dell'Analisi

L'analisi statica completa con domini infiniti si svolge in due fasi sequenziali:

1. **Ascending Phase (Fase Ascendente):** Si calcola la sequenza partendo da $\perp$ e utilizzando il widening (∇) nei punti di widening WP . Questa fase termina quando si raggiunge un post-punto

fisso x^∇ .

$$x_0 = \perp, \quad x_{i+1} = x_i \nabla F^\#(x_i)$$

2. **Descending Phase (Fase Discendente)**: Si parte dal risultato della fase precedente x^∇ e si utilizza il narrowing ($\triangle$) per raffinare la soluzione.

$$y_0 = x^\nabla, \quad y_{i+1} = y_i \triangle F^\#(y_i)$$

Il limite di questa sequenza $x^\triangle$ è un'approssimazione più precisa del least fixpoint reale.

Nota 14.1: Widening e narrowing

- **Complementarietà**: Widening e Narrowing sono due facce della stessa medaglia. Il primo “spinge” verso l’alto per garantire di coprire tutte le soluzioni (anche a costo di esagerare), il secondo “tira” verso il basso per eliminare l’eccesso.
- **Correttezza vs Precisione**: Il widening sacrifica la precisione per la terminazione. Il narrowing tenta di recuperare la precisione persa senza sacrificare nuovamente la terminazione.
- **Non monotonicità**: È interessante notare come il widening non sia monotono. Questo è controintuitivo rispetto alla maggior parte degli operatori sui reticoli, ma è necessario per “saltare” oltre i punti di accumulazione infiniti.